
Design and Characterization of a Three-Hall-Sensor Array for the Motion Detection and Damping Estimation of a Magnetic Pendulum

Authors: Aditya Ujjwal, Francesco Becchetti

Course: Laboratory of Sensors

Supervision: Prof. Alexandro Catini

Date: 06/07/2026

Abstract

This project set out to design, characterize, and relatively calibrate a three-sensor Hall-effect array to track the motion of a magnetic pendulum. A small neodymium magnet was mounted on a pendulum of length $L = 0.20$ m, and three analog 49E Hall sensors were positioned along its swing path in a linear array, spaced 3 cm apart (6 cm total span). Rather than using the sensors only to flag when the magnet crossed a point, the array was treated as an analog measurement system by analyzing offsets, noise, drift, relative sensor response, and the fusion of the three signals into one robust motion indicator.

The Arduino Uno R4 WiFi was used for data acquisition with its ADC configured at 14-bit resolution, corresponding to approximately 0.305 mV/count. This improved the quality of the recorded Hall signals compared with the lower-resolution configuration. An initial second-order Butterworth filter was tested for smoothing, but the final analysis relied mainly on higher ADC resolution and baseline correction, since the dominant visible limitation in the early readings was associated with quantization/resolution rather than high-frequency noise.

Each sensor had a different zero-field offset, so we corrected for this with baseline subtraction. Noise on the baseline was small (about 1.19–2.00 mV) compared to the ~ 1.5 V magnetic response, giving signal-to-noise ratios above 700 across all three sensors. From there, we used the cleaned signals to detect pendulum crossings, estimate the period, reconstruct the magnet's position with a weighted centroid method, and estimate damping from how the peak amplitude decayed over time.

The measured full period was 0.8766 s, close to the small-angle theoretical value of 0.8971 s but lower than the large-angle prediction of 0.9628 s for the measured 60-degree release. The damping analysis gave an effective decay coefficient near 0.11 s^{-1} . Since the sensors detect local magnetic field pulses rather than angle directly, this damping value should be read as an effective estimate rather than a strict mechanical damping constant. Altogether, the results show that a characterized Hall-sensor array can reliably track pendulum motion and support this kind of damping analysis.

1. Introduction

Measuring motion is one of the most common tasks in experimental physics and engineering. Depending on the application, this can be done using optical encoders, cameras, accelerometers, proximity sensors, or magnetic sensors. In this project, the motion of a pendulum was measured using magnetic sensing. A small neodymium magnet was attached to the pendulum bob, and a set of Hall-effect sensors was placed along the path of motion. As the magnet moved along the sensors, the changing magnetic field produced variations in the sensor output voltage.

The main purpose of the experiment was not simply to measure the period of a pendulum. Since this work was carried out for a Laboratory of Sensors course, the focus was placed on the sensor system itself: how the raw analog signals behave, how they can be relatively calibrated, and how useful information can be extracted from them. Real sensors do not produce ideal signals. They have offset, noise, drift, sensitivity differences, and limitations in resolution. For this reason, the raw data must be processed before it can be interpreted physically.

A three-sensor Hall array was used instead of a single central Hall sensor. A single sensor would be sufficient to detect when the magnet passes one point, but it would provide only a pulse-like crossing signal. With three sensors, the system can also observe how the magnetic response changes across position. This makes it possible to compare the individual sensors, compensate their offsets, normalize their responses, combine them into a fused signal, and estimate the local magnet position using a weighted centroid method. Therefore, the experiment becomes a sensor-array and signal-conditioning problem rather than only a simple pendulum timing experiment.

The pendulum itself was treated as a real mechanical system rather than an ideal one. In an ideal simple pendulum, the motion is lossless and the period depends mainly on the pendulum length and gravity. In the actual setup, however, energy is lost due to hinge friction, air drag, small misalignments, finite magnet size, and non-planar motion of the pendulum. These effects cause the oscillation amplitude to decay over time. The Hall sensors do not directly measure the angular displacement of the pendulum; instead, they detect local magnetic field pulses when the magnet passes near the array. Therefore, the crossing times can be used to estimate the period, while the decay of the magnetic peak envelope gives an effective indication of damping. This report describes the design of the Hall sensor array, the data acquisition system, the relative calibration and signal-conditioning procedure, and the analysis used to estimate pendulum period and damping from the processed sensor data.

2. Hardware Setup

2.1 Pendulum Structure

The mechanical part of the experiment consisted of a simple pendulum with a small neodymium magnet attached at its lower end. The magnet acted as the moving magnetic target, while the Hall sensors remained fixed below the pendulum path. As the pendulum oscillated, the magnet periodically passed above the sensor array and produced changes in the local magnetic field measured by the sensors.

The effective pendulum length was measured as the distance from the pivot point to the approximate center of the magnet, $L = 0.20$ m.

This length matters because it sets the pendulum's theoretical natural period. An ideal pendulum assumes a point-mass bob and a frictionless pivot, but neither holds exactly here: the magnet has finite size, the pivot has some mechanical friction, and the center of mass isn't perfectly defined. For this reason, L should be read as an effective measured length rather than an idealized one.

The vertical gap between the magnet and the sensor array was kept as small as practically possible without letting them touch. This mattered because the field from a small permanent magnet falls off quickly with distance; too large a gap would weaken the signal and make it more sensitive to noise and misalignment, while too small a gap risked the magnet striking the sensors mid-swing. The final spacing was chosen as a balance between these two constraints.

The pendulum was released by hand from a fixed angular displacement and left to swing freely. Because the release was manual, there was some uncertainty in the exact initial angle and starting conditions. Real losses, such as pivot friction, air resistance, and small misalignments, mean the system was not the ideal lossless pendulum described earlier in the introduction; these are the same mechanical non-idealities that later appear as amplitude decay in the damping analysis.

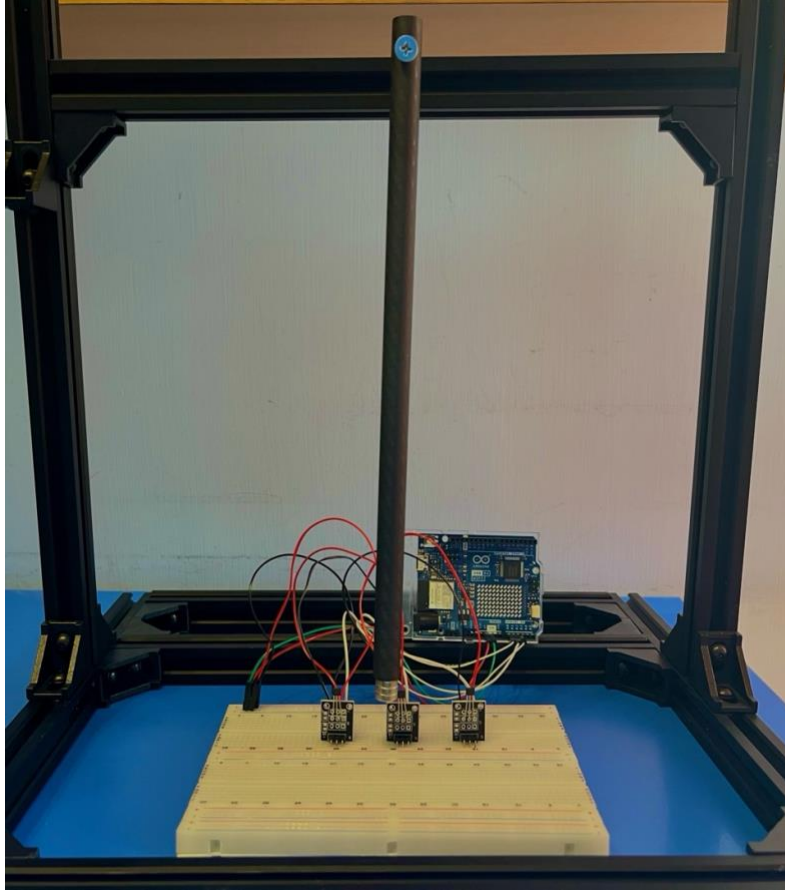


Figure 1. Physical setup of the magnetic pendulum showing the magnet at the end of the pendulum and the Hall sensor array along the path.

2.2 Hall Sensor Array

The sensing system used three 49E analog Hall-effect sensor modules. Each module had three terminals:

$$[+, -, S]$$

where (+) was connected to 5 V, (-) to ground, and (S) to an analog input of the Arduino.

The three sensors were connected as follows:

Hall sensor	Arduino pin
H1	A0
H2	A1
H3	A2

Using three sensors instead of a single central sensor provides spatial information about the magnetic bob. A single Hall sensor could detect when the magnet passes one point, but it would not indicate how the magnetic response changes across the path. With three sensors, the magnet produces different responses at H1, H2, and H3 depending on its local position. This allows the array to be used not only for crossing detection, but also for later signal fusion and local position estimation after relative calibration. The actual position reconstruction method is described in the signal-processing section.

The sensors were arranged in a straight line along the path of the pendulum. The spacing between adjacent sensors was approximately:

$$d = 0.03m$$

Therefore, the total span of the sensor array was approximately $0.06m$.

The central sensor was placed near the equilibrium position of the pendulum, and the other two sensors were placed symmetrically on either side.

The approximate sensor positions can be represented as:

$$\begin{aligned}x_1 &= -0.03m \\x_2 &= 0m \\x_3 &= +0.03m\end{aligned}$$

These position definitions are later used in the section 7 for the local position reconstruction of the magnet.

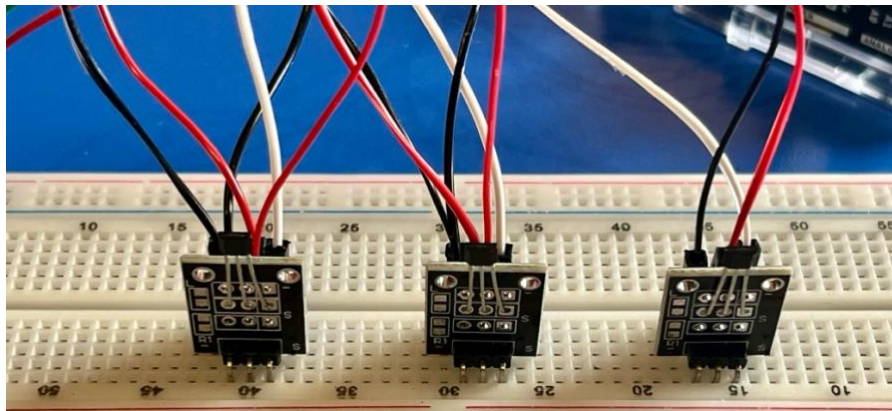


Figure 2. Three-Hall-sensor array with 3 cm spacing between adjacent sensors.

2.3 Sensor Type Choice

Analog 49E Hall sensors were chosen instead of digital on/off Hall sensors because the objective was to study the sensor response in detail. A digital Hall sensor only gives a binary output depending on whether the magnetic field exceeds a threshold, giving an output of either 0 or 1.

This would allow detection of crossing events, but it would not provide information about signal amplitude, sensor sensitivity, offset, noise, drift, or magnetic field variation.

In contrast, the analog 49E sensor gives a continuous voltage output. This allowed measurement of:

- baseline offset,
- voltage noise,
- signal amplitude,
- signal-to-noise ratio,
- sensor drift,
- sensitivity differences between sensors,
- normalized sensor response,
- fused magnetic response.

Therefore, the analog Hall sensor was more suitable, as it allowed both motion detection and sensor characterization.

3. Data Acquisition and Processing

The three Hall sensors produce analog voltage signals, so the first step of the experiment was to convert these voltages into reliable digital data. This was done using the Arduino Uno R4 WiFi, with the three sensor outputs connected to analog pins A0, A1, and A2. During each measurement, the Arduino sent one timestamp and three sensor readings to MATLAB in CSV format:

$$t_{\mu s}, H_1, H_2, H_3$$

A typical line of data contained the Arduino time in microseconds ($t_{\mu s}$), followed by the raw ADC readings (H_1, H_2, H_3) for each of the three Hall sensors. This simple format was useful because it allowed the same data file to be used later for plotting, relative calibration, noise analysis, position reconstruction, and damping estimation.

Before recording each run, the serial buffer was cleared so that old unread values from the Arduino were not mixed with the new measurement. This became important during testing, because otherwise MATLAB could begin plotting data that had already been sent before the actual experiment started.

3.1 Arduino Uno R4 WiFi and ADC Resolution

The analog Hall sensor outputs were read using the Arduino Uno R4 WiFi. The Arduino was configured to use 14-bit ADC resolution. Therefore, the digital output of the ADC lies in the range:

$$0 \leq N \leq 2^{14} - 1$$

$$0 \leq N \leq 16383$$

The observed reference voltage was $V_{ref} \approx 5V$

The voltage corresponding to one ADC count is:

$$\Delta V = \frac{V_{ref}}{16383}$$

$$\Delta V = 0.305mV/count$$

Thus, each ADC reading was converted into voltage using:

$$V_i(t) = \frac{N_i(t)}{16383} V_{ref}$$

where $N_i(t)$ is the raw ADC value of sensor (i).

The use of 14-bit resolution was important because the standard lower-resolution ADC mode gives a much larger voltage step. This resolution greatly reduced the staircase-like quantization behavior in the sensor readings.

3.2 Initial Filtering Attempt

During the first tests, a second-order Butterworth low-pass filter was tried as a possible way to clean the Hall sensor readings. This was a reasonable first approach, because low-pass filters are commonly used when a measurement contains unwanted high-frequency noise.

However, the Hall sensor signal in this experiment is not a smooth sinusoid. When the magnet passes over a sensor, the useful signal appears as a sharp pulse or dip. A strong low-pass filter can smooth these pulses, reduce their peak value, and slightly delay their timing. Since both the peak value and crossing time are important for the later analysis, heavy filtering was not ideal.

After switching the Arduino to 14-bit ADC mode, the signal quality improved significantly. This showed that the dominant visible roughness in the early plots was mainly related to ADC resolution and quantization, rather than strong high-frequency noise. This does not mean that the measurement had no noise. Sensor noise, ADC noise, supply variation, and environmental effects are still present, but their measured amplitude was small compared with the magnetic response.

For this reason, the final processing used baseline correction, drift compensation, and normalization instead of heavy filtering. A light smoothing operation was used only for plotting and peak detection, not to replace the original experimental data.

3.3 Serial Acquisition in MATLAB

The Arduino transmitted the data to MATLAB through serial communication at a baud rate of 230400.

This relatively high baud rate was selected so that timestamped readings from all three sensors could be transmitted without significant serial delay. In MATLAB, the incoming serial lines were read one by one and parsed as CSV data. Only valid lines containing four numerical values were stored. Any incomplete, non-numeric, or header line was ignored.

Before each acquisition, the serial buffer was flushed. This was necessary because old unread serial data could otherwise remain in the buffer and appear at the beginning of a new run. The MATLAB script also reset the local time axis at the beginning of each recording, so that the plots started from $t = 0$.

Two recordings were made:

1. **Baseline recording:** the pendulum was kept still or away from the sensors. This was used to measure sensor offset, noise, drift, and limit of detection.
2. **Motion recording:** the pendulum was released and allowed to oscillate freely over the Hall sensor array.

During the motion recording, the pendulum was held briefly before release. This created a short still section at the start of the data, making it easier to identify the beginning of the motion.

The MATLAB acquisition script (can be found in the Appendix B) was used to read serial data, store raw ADC readings, convert them to voltage, and export processed CSV files.

4. Sensor Model

Each Hall sensor output can be modelled as the sum of the baseline offset, the measurement noise and the detected magnetic field component:

$$N_i(t) = b_i(t) + g_i B_i(t) + n_i(t)$$

where:

- $N_i(t)$ is the raw ADC reading,
- $b_i(t)$ is the baseline offset,
- g_i is the sensitivity or gain of the sensor,
- $B_i(t)$ is the magnetic field component detected by the sensor,
- $n_i(t)$ is the measurement noise.

This model shows that the raw ADC reading is not only the magnetic signal. It also contains the sensor's own offset, small random noise, and possible slow baseline drift. Therefore, the raw readings cannot be compared directly until the baseline and sensor-specific effects are corrected.

In this experiment, the chosen magnet orientation produced a decrease in Hall sensor output when the magnet approached the sensor. This means that the useful magnetic response appeared as a downward dip in the raw ADC signal. The conversion of these voltage dips into positive magnetic response peaks is discussed in the next section.

5. Sensor Characterization

5.1 Calibration Approach

In this project, the Hall sensors were not calibrated as absolute magnetic-field meters. An absolute magnetic-field calibration would require a known reference magnetic field, such as a calibrated Helmholtz coil or a gaussmeter, so that the ADC output could be converted directly into magnetic flux density in tesla or gauss. This equipment was not available during the experiment.

Instead, the sensors were calibrated and characterised for the purpose of relative magnetic motion detection. The main aim was to identify when the magnet passed near each Hall sensor and to compare the responses of H1, H2, and H3 within the same sensor array. Therefore, the calibration procedure focused on the experimentally important quantities: baseline offset, noise level, limit of detection, peak magnetic response, drift, and relative sensitivity.

First, a stationary baseline recording was taken with the pendulum still and the magnet away from the active sensing region. The median baseline value of each sensor was used as its offset value. This was necessary because the three Hall sensors did not have identical zero-field outputs. The corrected magnetic response was then calculated relative to this baseline, allowing the downward voltage dips caused by the magnet polarity to be converted into positive response peaks.

The calibration was therefore a **relative sensor calibration**, not an absolute magnetic-field calibration. It allowed the raw ADC readings to be transformed into corrected and normalised response signals suitable for peak detection, sensor fusion, and weighted-centroid position estimation. This was sufficient for the objective of the experiment because the analysis depended on the timing, relative amplitude, and spatial distribution of the Hall responses rather than on the absolute value of magnetic flux density.

5.2 Raw ADC Data and Voltage Conversion

The raw Hall sensor readings were acquired directly from Arduino analog inputs A0, A1, and A2.

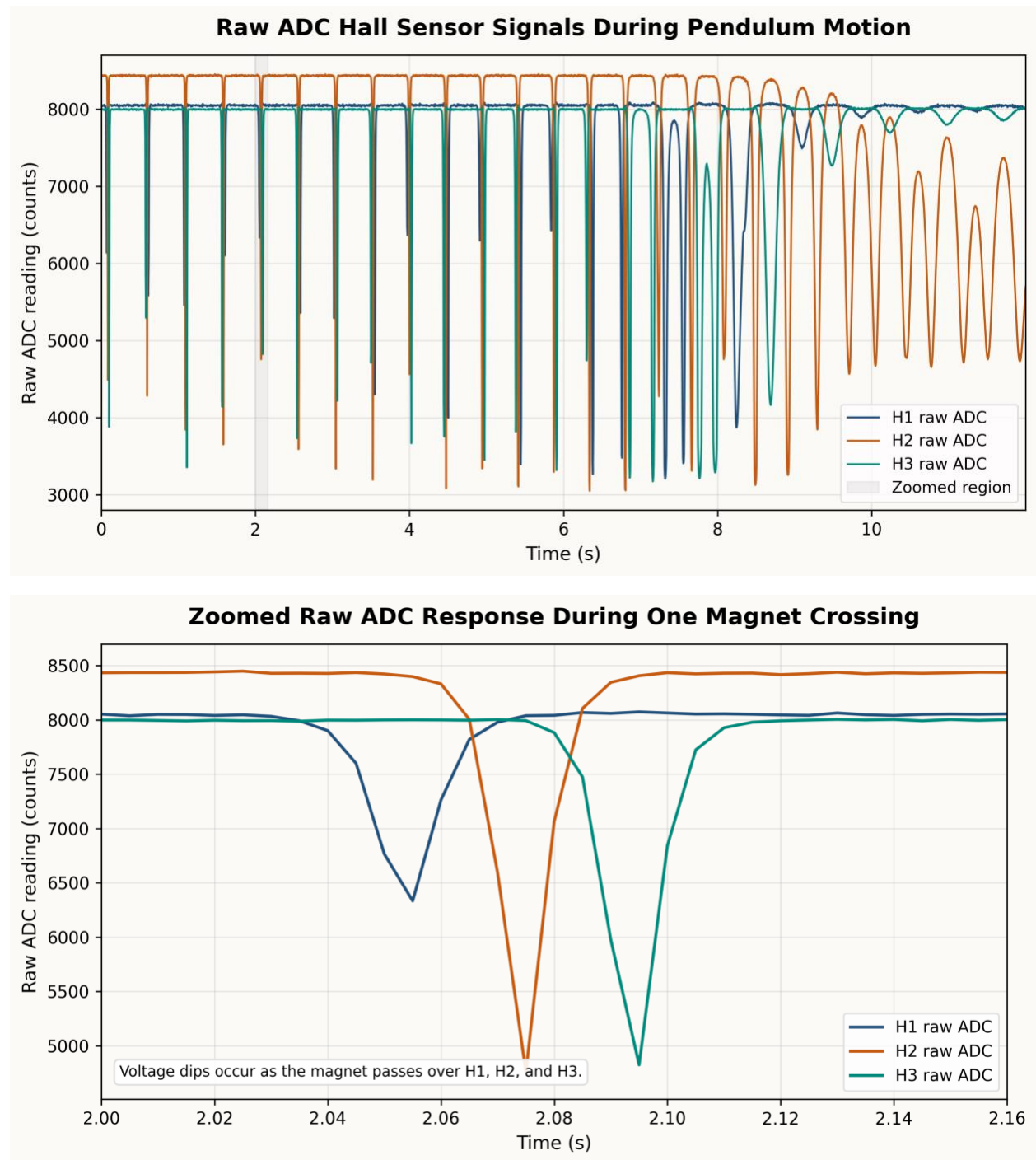


Figure 3. Raw Hall sensor readings during pendulum motion.

5.3 Baseline Offset and Polarity Correction

A baseline recording was taken with no pendulum motion. This was used to estimate the zero-field output of each Hall sensor.

The measured median baseline values were:

Sensor	Baseline ADC count	Baseline voltage
H1	8048	2.46 V
H2	8435	2.57 V
H3	7996	2.44 V

The central sensor H2 had a noticeably higher baseline than H1 and H3. This is expected for analog Hall sensor modules because each sensor has its own offset, module electronics, and small sensitivity differences. If this offset were not removed, H2 would appear larger than the other sensors even when no magnet was passing over it.

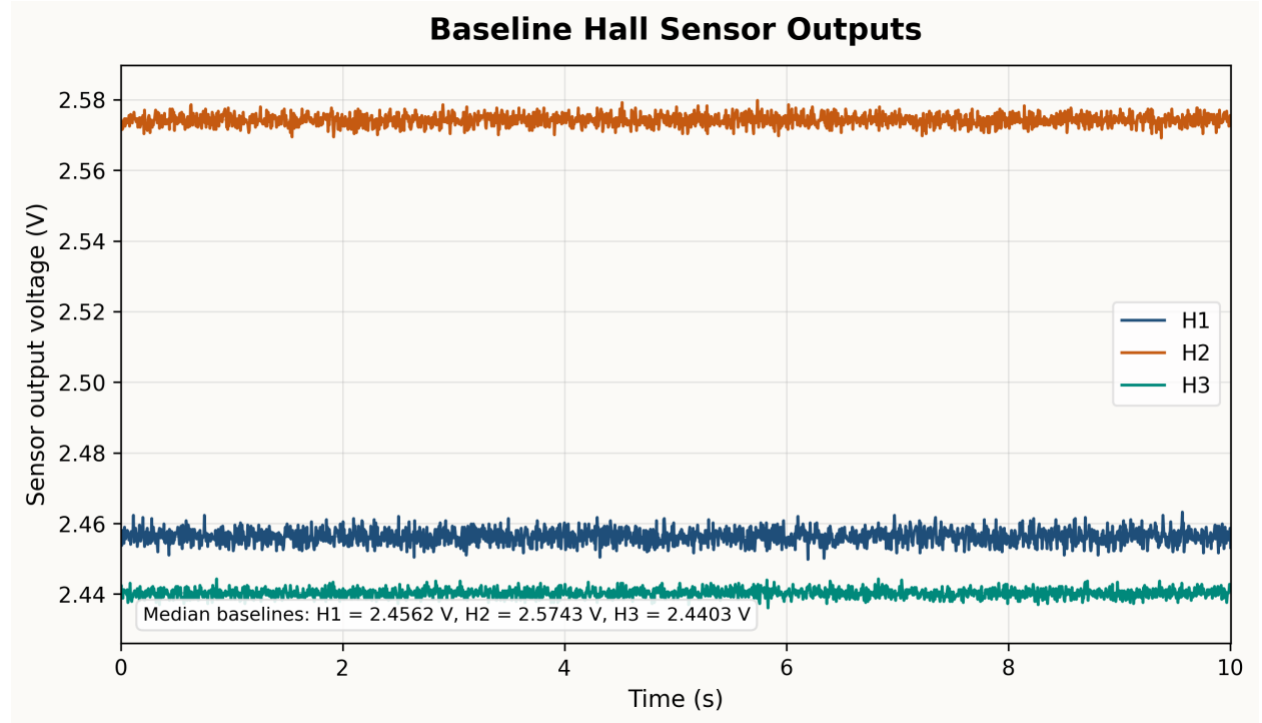


Figure 4. Baseline Hall sensor readings showing different zero-field offsets for H1, H2, and H3.

Because the magnet polarity caused the Hall readings to decrease near the magnet, the magnetic response was calculated as:

$$S_i(t) = b_i - N_i(t)$$

where b_i is the baseline value and $N_i(t)$ is the raw ADC reading for sensor i .

Small negative values can occur due to baseline noise when the magnet is far from the sensor. These values do not represent useful magnetic response, so they were clipped to zero:

$$S_i(t) = \max(b_i - N_i(t), 0)$$

This converted downward voltage dips into positive magnetic response peaks.

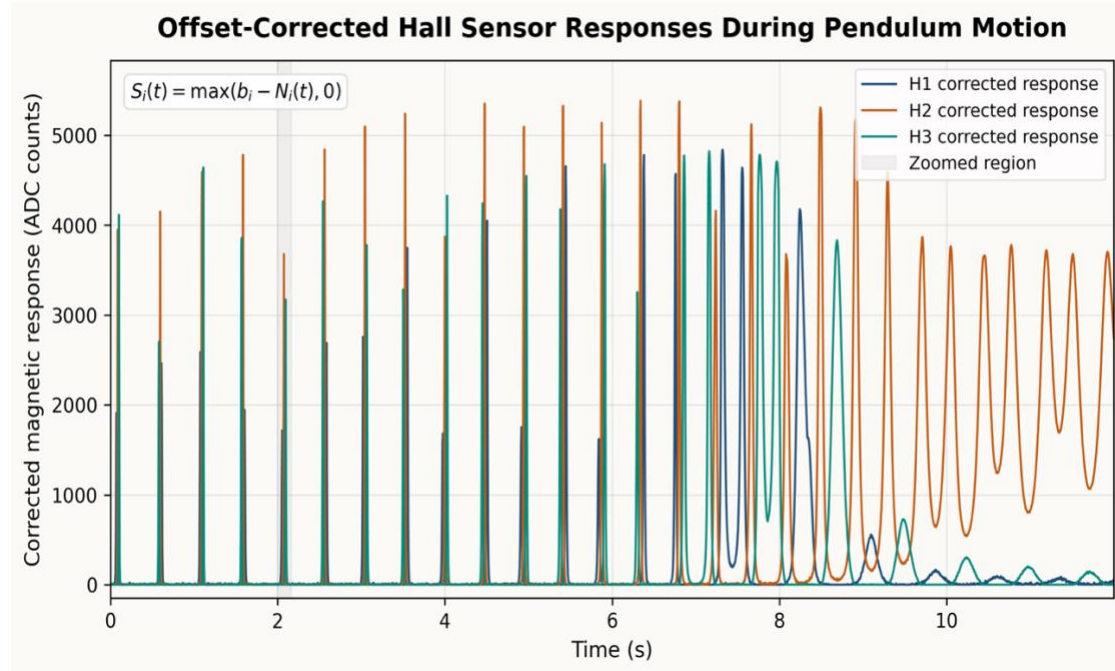


Figure 5. Offset-corrected Hall sensor responses after baseline subtraction and zero clipping.

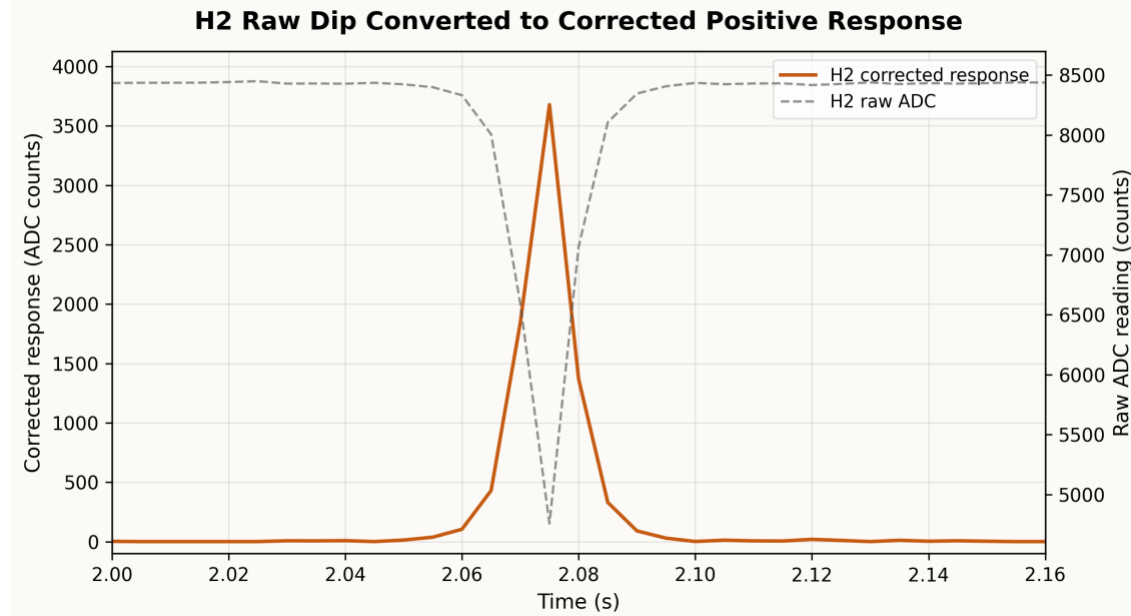


Figure 6. Polarity correction example for H2. The raw ADC signal decreases when the magnet approaches, and baseline subtraction converts this dip into a positive magnetic response peak.

5.4 Noise and Limit of Detection

The noise of each sensor was estimated from the baseline recording by computing the standard deviation:

$$\sigma_i = \text{std}(N_i)$$

The measured noise values were:

Sensor	Noise std.	Noise voltage
H1	6.56 counts	2.003 mV
H2	5.12 counts	1.562 mV
H3	3.89 counts	1.188 mV

These noise values are small (in the mV range) compared with the magnetic response. The baseline noise was only a few ADC counts, while the magnet produced responses of several thousand counts (in the volt range).

The limit of detection was estimated using the $3\sigma_i$ criterion:

$$LOD_i = 3\sigma_i$$

The measured limits of detection were:

Sensor	LOD	LOD voltage
H1	19.69 counts	6.010 mV
H2	15.36 counts	4.687 mV
H3	11.68 counts	3.564 mV

The magnetic responses were much larger than the LOD values, meaning that the magnet was detected very reliably.

The LOD reported here is an ADC-count/voltage detection limit, not an absolute magnetic-field limit. A magnetic-field LOD in mT would require an independent calibration of each Hall sensor against known magnetic field values.

5.5 Peak Magnetic Response, Sensitivity and Signal to Noise Ratio

The peak magnetic responses were:

Sensor	Peak magnetic response	Voltage response
H1	4840 counts	1477.1 mV
H2	5384 counts	1643.2 mV
H3	4822 counts	1471.6 mV

H2 showed the largest peak response, which may be due to its position relative to the magnet path, slightly different sensor sensitivity, or mechanical alignment.

In this report, sensitivity is therefore used in a relative sense. The listed peak responses characterize how strongly each module reacted to the experimental magnet and geometry; they should not be interpreted as absolute sensitivities in mV/mT.

For comparison, the peak response was hundreds of times larger than the noise level. This confirms that the sensor system had strong detection capability.

The signal-to-noise ratio was calculated as:

$$SNR_i = S_{i,max} / \sigma_i$$

The measured SNR values were:

Sensor	SNR
H1	737.30
H2	1051.75
H3	1238.65

These values are very high and show that the magnetic signal was extremely strong compared with the baseline noise. This also supports the decision not to use a strong Butterworth filter in the final analysis.

5.6 Drift and Baseline Compensation

Sensor drift was estimated by fitting a straight line to the baseline data:

$$b_i(t) \approx b_{i0} + m_i t$$

where m_i is the drift slope.

The measured drift slopes were:

Sensor	Drift slope	Drift slope voltage
H1	0.0137 counts/s	0.0042 mV/s
H2	0.0316 counts/s	0.0096 mV/s
H3	0.0363 counts/s	0.0111 mV/s

These drift values are extremely small compared with the magnetic response. For example, the maximum drift slope was approximately 0.0111 mV/s, while the magnetic response was on the order of 1.47 V to 1.64 V.

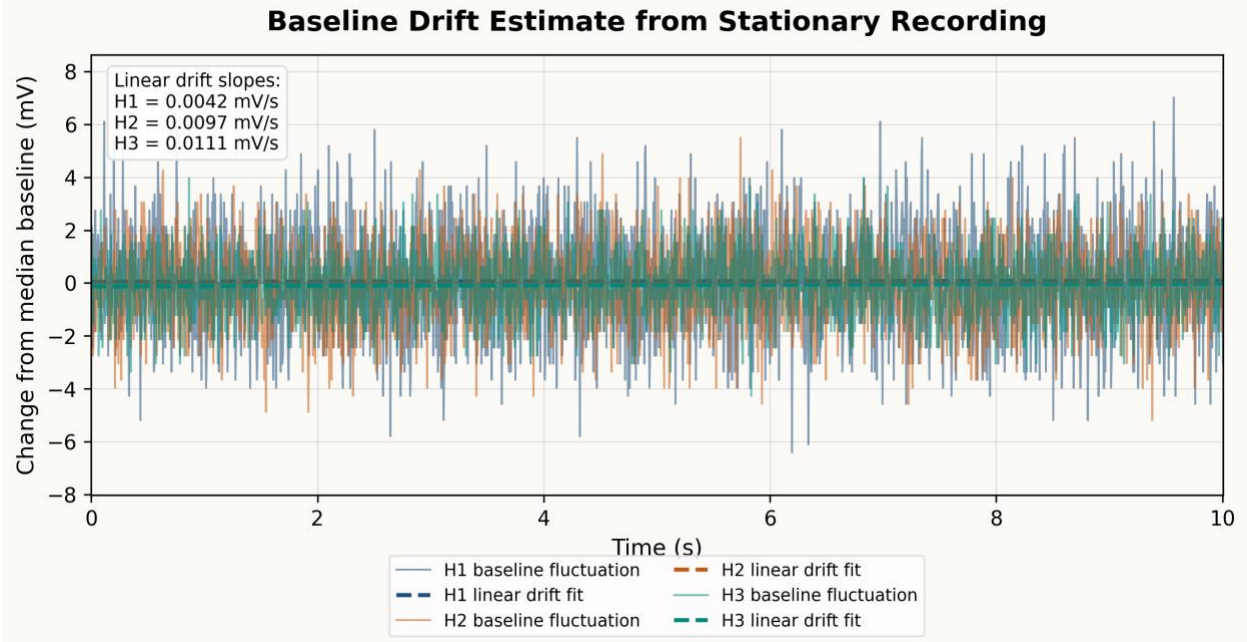


Figure 7. Baseline drift estimate from stationary Hall sensor recording. The sensor outputs are shown relative to their median baseline values, with linear drift fits. The drift slopes are very small compared with the magnetic response.

Thus, drift was negligible during a single 12-second measurement run. However, drift compensation was still included in the processing pipeline to make the sensor system more robust during longer recordings or when slow baseline changes are caused by temperature, supply-voltage variation, or environmental effects.

A slow adaptive baseline estimate was implemented using:

$$b_i[k] = \alpha b_i[k - 1] + (1 - \alpha)N_i[k]$$

with: $\alpha = 0.999$

The baseline was updated only when the total magnetic response was below a threshold, meaning that the magnet was far from the sensor array. This prevented the real magnetic signal from being accidentally removed as drift.

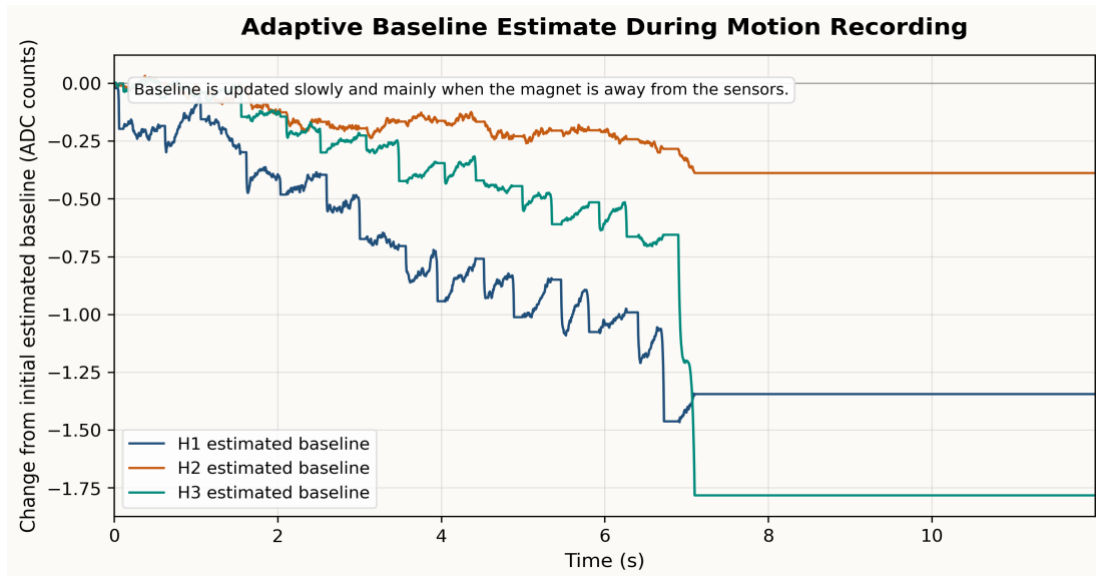


Figure 8. Adaptive baseline correction during the motion recording. The estimated baseline changes slowly and is updated mainly when the magnet is away from the Hall sensor array.

5.7 Sensitivity Normalization and Sensor Fusion

The sensors had slightly different peak responses. Therefore, each signal was normalized by its own maximum:

$$S_{i,\text{norm}}(t) = \frac{S_i(t)}{S_{i,\text{max}}}$$

This placed all three responses on a common dimensionless scale from 0 to 1. The comparison then depends mainly on the relative magnetic response at each sensor, not on the fact that one module may naturally produce a larger voltage than the others.

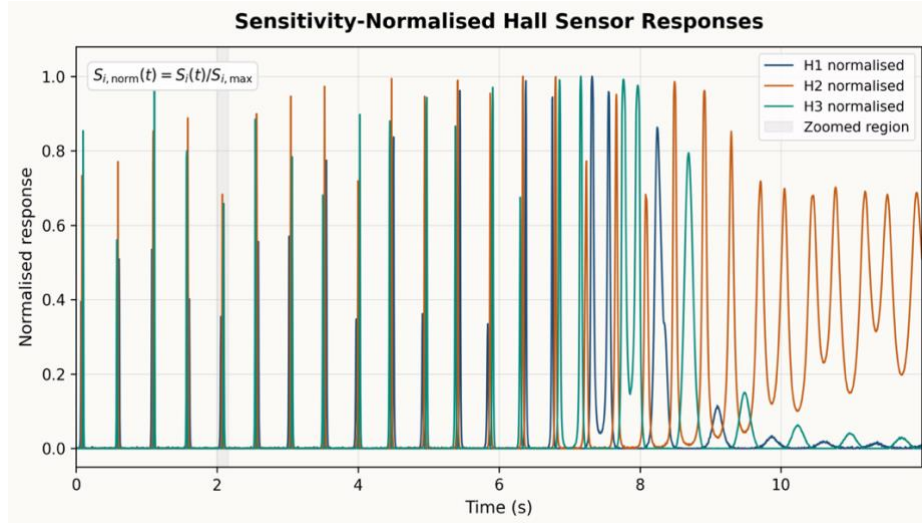


Figure 9. Sensitivity-normalized Hall sensor responses during pendulum motion. Each corrected sensor signal was divided by its own maximum response so that H1, H2, and H3 could be compared on the same relative scale.

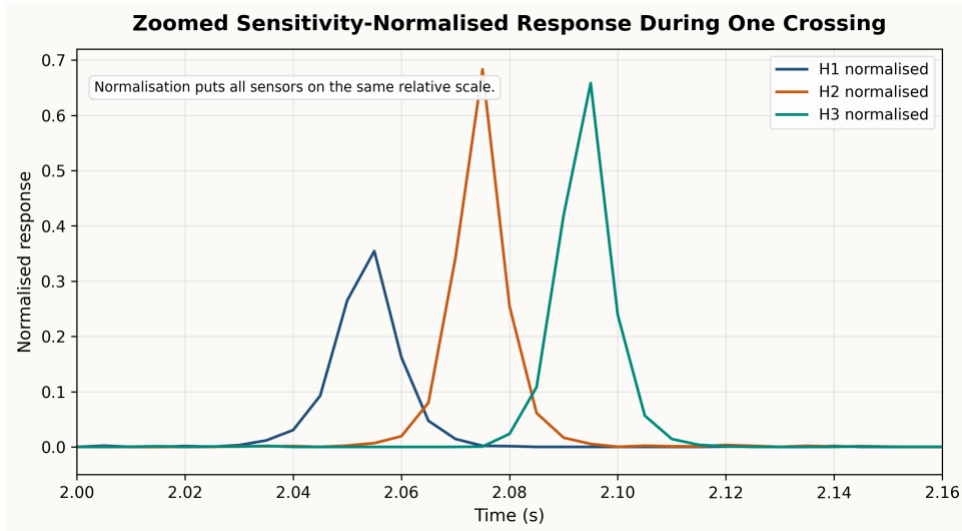


Figure 10. Zoomed normalized Hall responses during one magnet crossing.

A fused magnetic response was then computed to obtain one global indicator of the magnet passing through the sensor array:

$$S_{\text{total}}(t) = S_1(t) + S_2(t) + S_3(t)$$

and for normalized signals:

$$S_{\text{total, norm}}(t) = S_{1, \text{norm}}(t) + S_{2, \text{norm}}(t) + S_{3, \text{norm}}(t)$$

The fused response gives a single indicator of the magnet passing near the sensor array. It is useful for crossing detection, period estimation, and damping-envelope analysis because it uses information from all three sensors instead of relying on only one channel. However, because the magnet passes over H1, H2, and H3 separately, one physical crossing can produce several local peaks; for timing analysis these peaks must be interpreted carefully.

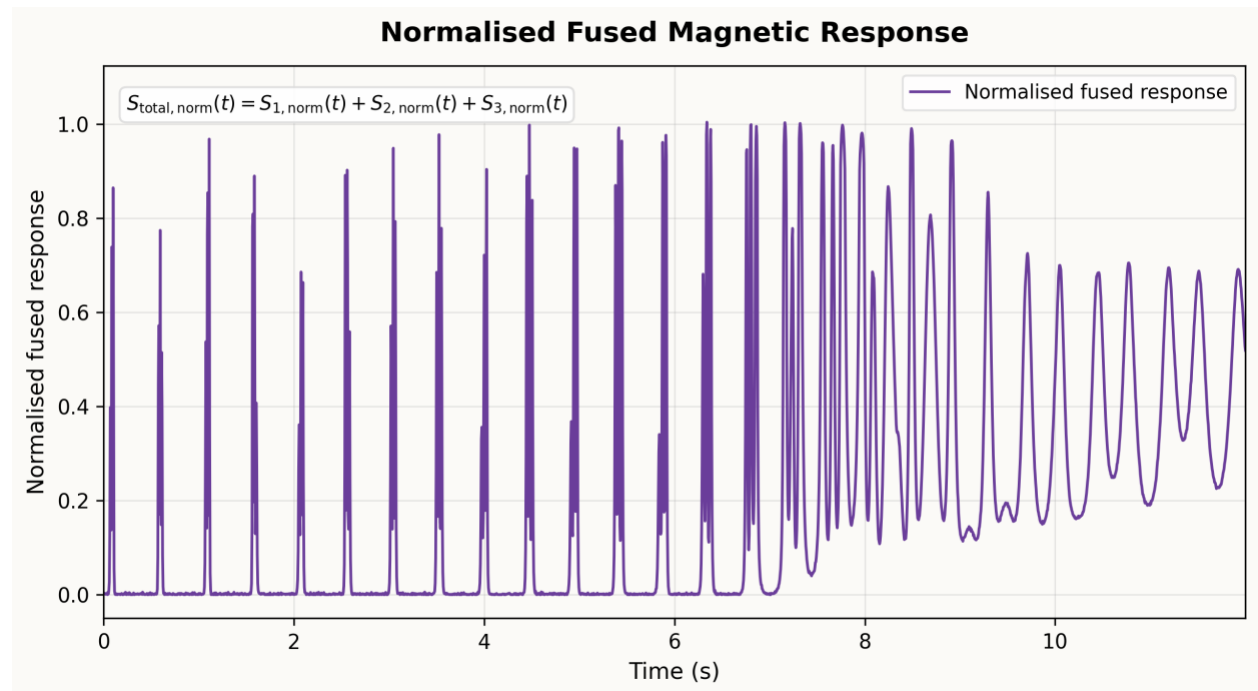


Figure 11. Fused magnetic response from the three-sensor array.

6. Pendulum Theory

For an ideal simple pendulum without damping, the nonlinear equation of motion is:

$$\ddot{\theta} + \frac{g}{L} \sin \theta = 0$$

For small angles:

$$\sin \theta \approx \theta$$

and the equation becomes:

$$\ddot{\theta} + \frac{g}{L} \theta = 0$$

The small-angle period is then:

$$T_0 = 2\pi \sqrt{\frac{L}{g}}$$

For:

$$L = 0.20\text{m}$$

$$g = 9.81\text{m/s}^2$$

$$T_0 = 0.8971\text{s}$$

However, the pendulum was released from a relatively large angle, approximately 60° .

Therefore, the small-angle approximation is not fully valid. The nonlinear large-angle period is:

$$T(\theta_0) = 4 \sqrt{\frac{L}{g}} K\left(\sin^2 \frac{\theta_0}{2}\right)$$

where K is the complete elliptic integral¹ of the first kind and (θ_0) is the release angle in radians.

For $\theta_0 \approx 60^\circ$, the large-angle corrected period is:

$$T_{60^\circ} = 0.9628\text{s}$$

¹ The elliptic integral $K(\sin^2(\theta_0/2))$ is evaluated using the parameter convention $K(m)$, $m = k^2$ (consistent with standard numerical libraries such as SciPy/MATLAB's `ellipke`), not the modulus convention $K(k)$ common in some classical mechanics texts. This convention was used consistently to obtain $T(60^\circ) = 0.9628$ s.

7. Local Position Reconstruction Using the Sensor Array

After the Hall sensor signals were corrected and normalized, the three-sensor array was used to estimate the local position of the magnet in one dimension as it passed through the sensing region. This analysis was included because one of the main advantages of using three Hall sensors instead of one is that the system can provide spatial information, not only crossing detection.

A single Hall sensor can show that the magnet has passed near one point. However, it cannot indicate whether the magnet is moving from left to right, right to left, or where it is located within a small sensing region. With three sensors placed along the pendulum path, the relative strength of the three sensor responses changes as the magnet moves across the array. This allows the local magnet position to be estimated from the distribution of the Hall responses.

The sensor positions were defined as $[x_1, x_2, x_3] \equiv [-3, 0, +3]$ cm.

The central sensor was placed approximately below the equilibrium position of the pendulum. These position definitions were used only for the local reconstruction of the magnet position within the sensor array.

The position estimate was calculated using the normalized Hall sensor responses:

$$S_{1,norm}(t), S_{2,norm}(t), S_{3,norm}(t)$$

The normalized signals were used instead of the raw corrected signals because the three sensors did not have the same peak response. Without normalization, the sensor with the largest amplitude could bias the position estimate even when the magnet was not physically closest to that sensor. Normalization placed the three responses on a common scale from 0 to 1, making the centroid method more balanced.

The estimated local position was obtained using a weighted centroid equation:

$$x_{est}(t) = \frac{x_1 S_{1,norm}(t) + x_2 S_{2,norm}(t) + x_3 S_{3,norm}(t)}{S_{1,norm}(t) + S_{2,norm}(t) + S_{3,norm}(t)}$$

This equation works like a center-of-response calculation. Each sensor position is weighted by the strength of its response at that instant. If H1 gives the strongest response, the estimated position moves toward -0.03 m. If H2 dominates, the estimate moves toward 0 m. If H3 gives the strongest response, the estimate moves toward $+0.03$ m.

The total normalized response was also calculated as:

$$S_{total,norm}(t) = S_{1,norm}(t) + S_{2,norm}(t) + S_{3,norm}(t)$$

This value was used to decide whether the centroid estimate was meaningful. When the magnet is far from the sensor array, all three Hall responses are close to zero. In that case, the denominator of the centroid equation becomes very small, and the calculated position can become unstable or dominated by noise. Therefore, the position estimate was interpreted only when the total response was sufficiently large, meaning that the magnet was actually passing through the sensing region.

The estimated horizontal position was then converted into an approximate local angular displacement using the pendulum geometry, $x = L \sin \theta$

Rearranging gives: $\theta_{est} = \sin^{-1} \left(\frac{x_{est}(t)}{L} \right)$

where L is the effective pendulum length (0.20 m).

Since the outer sensors are located at: $x = \pm 0.03m$

the angular range corresponding to the physical sensor span is:

$$\theta = \sin^{-1} \left(\frac{0.03}{0.20} \right)$$

Therefore, the three-sensor array can reconstruct the magnet position only within approximately:

$$-8.6^\circ \leq \theta_{est} \leq +8.6^\circ$$

This is much smaller than the measured initial release angle of approximately 60° . For this reason, the position reconstruction does not represent the full angular motion of the pendulum from one extreme to the other. Instead, it gives a local estimate of the magnet position only when the magnet passes near the center of the swing, where the Hall sensors are located.

This distinction is important. The centroid method does not fail because of ADC saturation or because the sensors stop working. It is limited by geometry. The physical sensor array covers only a 6 cm span, so when the magnet moves outside this span, the three sensors no longer contain enough information to reconstruct the true position. In this case, the estimated position tends to remain close to the outer sensor positions, giving the appearance of clipping near -3 cm or $+3$ cm.

The reconstructed angle should also be interpreted carefully. The Hall sensors do not directly measure angular displacement. They measure local magnetic field strength, which depends on magnet position, distance from the sensor, magnet orientation, and alignment. The conversion from x_{est} to θ_{est} assumes that the estimated horizontal position represents the magnet's position along the pendulum path. This approximation is most reasonable near the central sensing region, where the motion is almost horizontal over the small array span.

Several practical factors can affect the accuracy of the reconstructed position:

- the magnet may not pass exactly above the center line of the sensors,
- the magnet-sensor height may change slightly during motion,
- the pendulum may twist or move slightly out of plane,
- the three Hall sensors may have different sensitivities,
- the magnetic field of the magnet is not a perfect point source,
- the response of each sensor is not perfectly linear with distance.

These effects mean that the centroid result should be treated as an approximate local position estimate, not as a precise measurement of the complete pendulum trajectory.

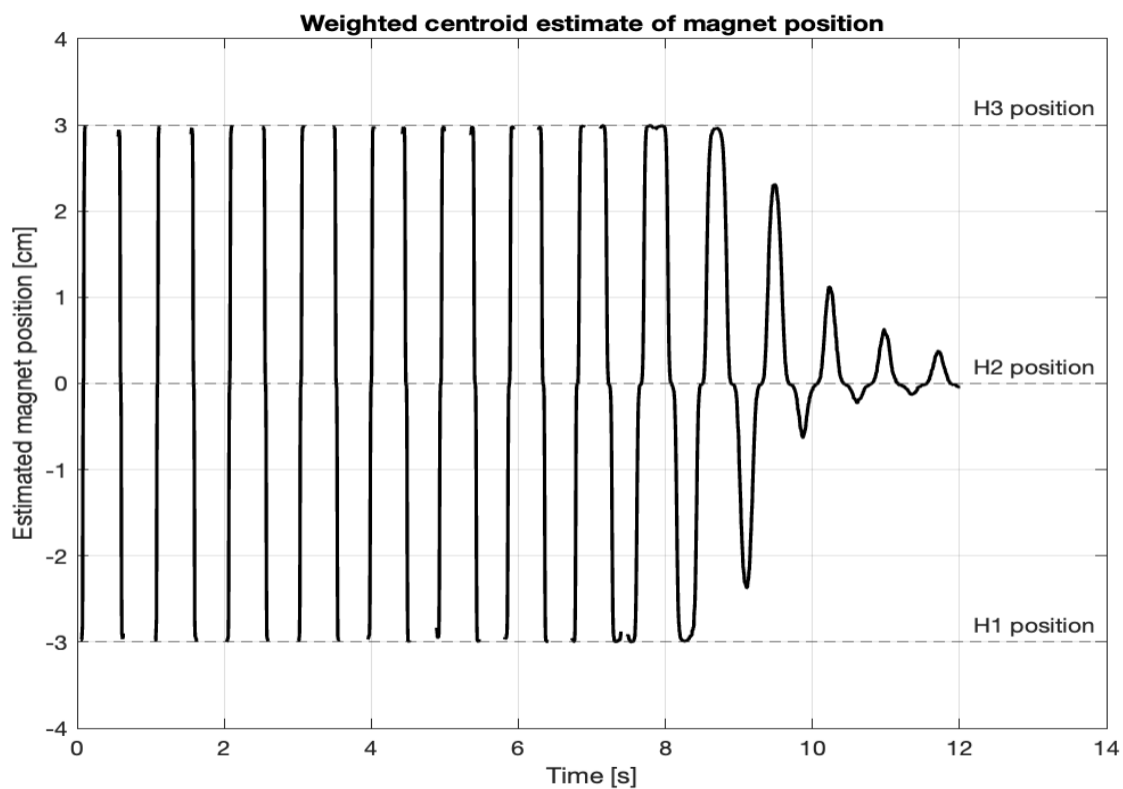


Figure 12. Weighted-centroid estimate of local magnet position. The position is meaningful only when the magnet passes through the Hall sensor array.

Snapshot Sequence of Weighted-Centroid Position Reconstruction

Selected static frames from the animated magnetic-bob tracking result.

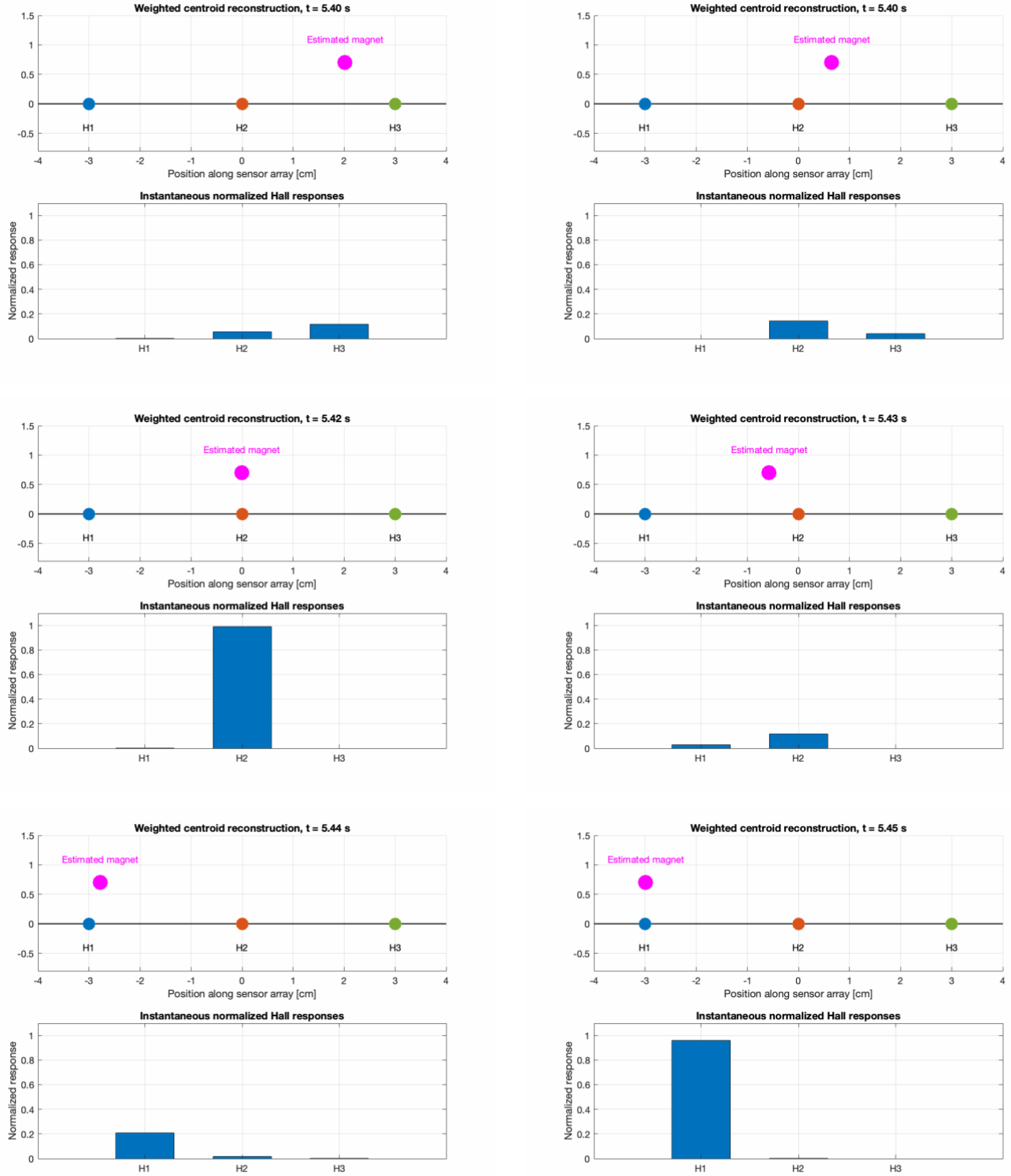


Figure 13. Snapshot sequence of weighted-centroid position reconstruction. The upper panel in each frame shows the estimated horizontal position of the magnet along the sensor array, while the lower panel shows the instantaneous normalized responses of

H1, H2, and H3 used to compute the centroid. As the bob passes through the sensing region, the dominant response shifts between sensors and the estimated x-position moves accordingly.

Note on axes. Only the x-coordinate in the upper panel has physical meaning. It represents the estimated one-dimensional magnet position along the sensor array, with sensor centres located at approximately -3 cm, 0 cm and +3 cm. The y-axis is not a measured vertical displacement or second spatial coordinate; it is only a visual offset used to display the marker and labels clearly. Therefore, this figure should be interpreted as 1D tracking along the sensor axis.

Purpose of the snapshot sequence. The original output was an animation, so these frames provide a static summary suitable for the PDF report. They show how the centroid estimate is pulled toward the sensor with the strongest response and moves between neighbouring sensors when their responses overlap.

8. Damping Estimation

8.1 Mathematical Modeling in Simulink

After the experimental Hall sensor data were corrected, normalized, and used for local position reconstruction, a simplified Simulink model was developed to support the interpretation of the pendulum motion and damping behavior. The purpose of this model was not to exactly reproduce every experimental detail, but to create a digital representation of the main physical effects present in the system: nonlinear pendulum motion, energy loss due to damping, and the local response of the Hall sensor array.

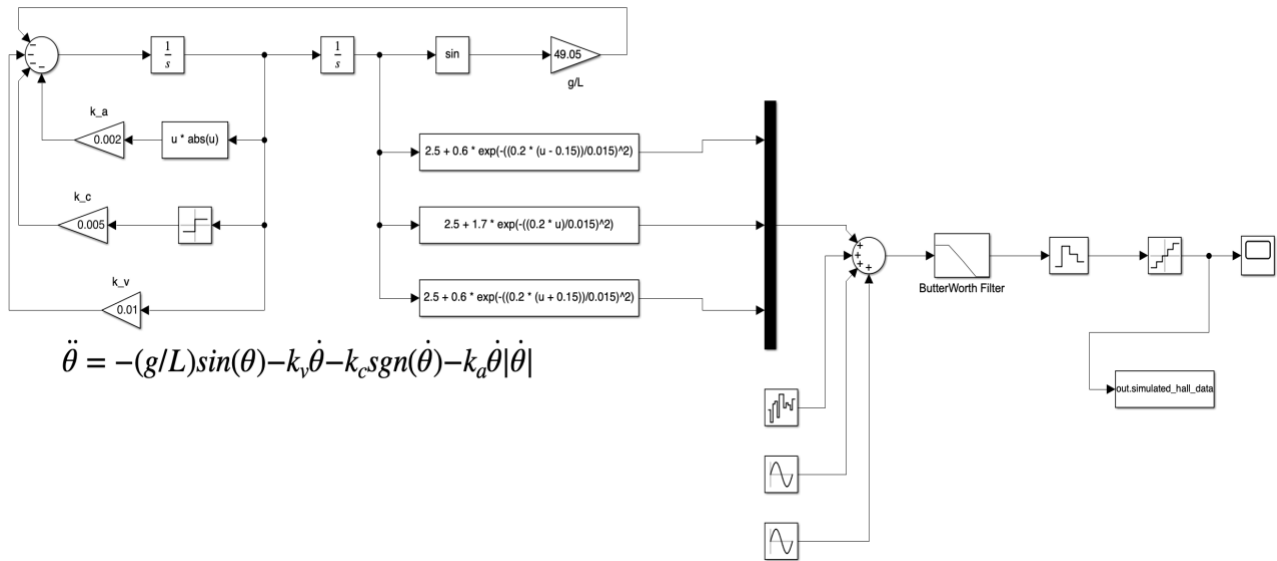
The experimental setup was not an ideal simple pendulum. The pivot had friction, the magnet and bob experienced air resistance, and the pendulum was released from a measured initial angle of approximately 60° , where the small-angle approximation is not fully valid. Therefore, a nonlinear model was developed and analyzed, governed by the equation:

$$\ddot{\theta} = -\frac{g}{L} \sin \theta - k_v \dot{\theta} - k_c \operatorname{sgn}(\dot{\theta}) - k_a \dot{\theta} |\dot{\theta}|$$

where:

- $k_v \dot{\theta}$ represents viscous-like damping,
- $k_c \operatorname{sgn}(\dot{\theta})$ represents Coulomb friction at the hinge,
- $k_a \dot{\theta} |\dot{\theta}|$ represents quadratic air drag.

These coefficients are used as effective parameters per unit moment of inertia in the normalized angular equation.



8.2 Experimental Peak Detection from Hall Data

The damping and period analysis was carried out using the processed Hall sensor data. Since the Hall sensors do not directly measure angular displacement, the analysis was based on the fused magnetic response of the three-sensor array. The fused response combines the corrected Hall signals:

$$S_{\text{total}}(t) = S_1(t) + S_2(t) + S_3(t)$$

or, when using normalized responses:

$$S_{\text{total,norm}}(t) = S_{1,\text{norm}}(t) + S_{2,\text{norm}}(t) + S_{3,\text{norm}}(t)$$

The detected peaks in this fused signal represent times when the magnet passed through the sensor region. These peaks were used for two purposes:

1. to estimate the oscillation period from the time spacing between crossings;
2. to estimate damping from the decay of the peak amplitudes.

The Hall peak amplitude is not the same as the true angular amplitude of the pendulum. It depends on the distance between the magnet and the sensors, alignment, magnet orientation, and the local magnetic field distribution. Therefore, the damping obtained from this method is treated as an **effective damping estimate**, not an exact mechanical damping constant.

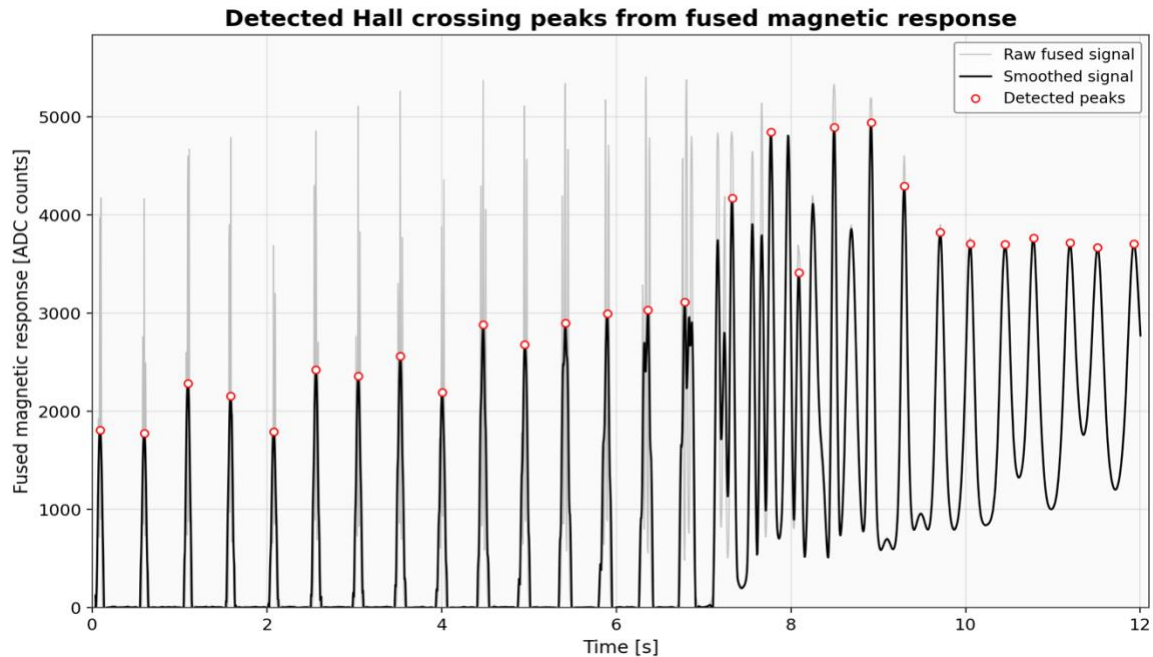


Figure 14. Detected Hall crossing peaks used for period and damping estimation.

8.3 Period Measurement from Hall Crossing Peaks

The time difference between consecutive detected Hall crossing peaks was used to estimate the half-period of the pendulum. This is because a pendulum passes through the central sensing region twice during one complete oscillation: once in one direction and once in the opposite direction.

If the mean time between consecutive crossings is $\Delta t_{\text{crossing}}$, then the full period is:

$$T_{\text{measured}} = 2\Delta t_{\text{crossing}}$$

The measured values were:

Quantity	Value
Number of detected Hall crossing peaks	28
Mean half-period	0.4383 s
Standard deviation of half-period	0.0619 s
Measured full period	0.8766 s
Scatter in full-period estimates	± 0.1238 s
Measured frequency	1.1408 Hz

Thus,

$$T_{\text{measured}} = 0.8766 \pm 0.1238 \text{ s}$$

The theoretical small-angle period for $L = 0.20$ m was:

$$T_0 = 0.8971 \text{ s}$$

The percentage error relative to the small-angle theory was:

$$\frac{0.8766 - 0.8971}{0.8971} \times 100 = -2.29\%$$

For the measured release angle of approximately 60° , the large-angle theoretical period was:

$$T_{60^\circ} = 0.9628 \text{ s}$$

The percentage error relative to the large-angle theory was:

$$\frac{0.8766 - 0.9628}{0.9628} \times 100 = -8.96\%$$

The measured period was close to the small-angle theoretical value but lower than the large-angle theoretical value. Since a larger release angle should increase the period, this difference is likely related to the experimental peak-detection method and the sensor geometry. The fused Hall signal can produce more than one peak during a single physical crossing because the magnet passes over H1, H2, and H3 separately. This can cause the detected crossing interval to

be slightly underestimated if peak detection is not perfectly matched to the physical centre crossings.

Other possible sources of difference include uncertainty in the effective pendulum length, small uncertainty in the release angle despite protractor measurement, finite magnet size, sensor spacing, pivot friction, air drag, and slight non-planar or twisting motion.

8.4 Exponential Damping Envelope

After the Hall crossing peaks were detected, their amplitudes were used to estimate the decay envelope. Since the Hall sensors measure local magnetic field pulses rather than direct angular displacement, the peak envelope was treated as a proxy for the decay of the pendulum motion.

The exponential damping model was:

$$A(t) = A_0 e^{-\lambda t}$$

where $A(t)$ is the peak envelope, A_0 is the initial envelope amplitude, and λ is the decay coefficient.

Taking the natural logarithm gives:

$$\ln A(t) = \ln A_0 - \lambda t$$

Therefore, λ was obtained by fitting a straight line to $\ln A(t)$. The exponential fit gave:

$$\lambda = 0.1058 \text{ s}^{-1}$$

with:

$$R^2 = 0.7111$$

The value of R^2 shows that the exponential model captured the general decay trend, but it did not explain all variation in the peak amplitudes. This is expected because Hall peak heights are affected not only by damping, but also by small changes in magnet path, sensor alignment, and the fact that different sensors contribute to the fused signal.

8.5 Effective Damping Ratio and Viscous Coefficient

For a lightly damped oscillator, the damping ratio can be estimated approximately using:

$$\zeta \approx \frac{\lambda}{\omega}$$

Different choices of angular frequency were used to check the sensitivity of the result.

The equivalent damping ratios were:

Method	Damping ratio
Using $\omega_0 = \sqrt{g/L}$	0.0151
Using measured angular frequency	0.0148
Using large-angle angular frequency	0.0162

All three values are close to each other, which suggests that the damping estimate is reasonably consistent.

For an exponential decay model written in the form expected for a lightly damped second-order oscillator, the effective viscous coefficient per unit inertia can be estimated as:

$$k_v = 2\lambda$$

Therefore:

$$k_v = 2(0.1058)$$

$$k_v = 0.2116 \text{ s}^{-1}$$

This value should be interpreted as an effective damping coefficient per unit inertia. It is not a direct measurement of a pure mechanical damping constant because the Hall peak envelope is an indirect measurement of motion amplitude.

8.6 Logarithmic Decrement Estimate

A second damping estimate was obtained using the logarithmic decrement method. For successive peak amplitudes, the logarithmic decrement is:

$$\delta = \ln \left(\frac{A_n}{A_{n+1}} \right)$$

Using the experimental Hall peak amplitudes, the average logarithmic decrement was:

$$\delta = 0.0902 \pm 0.0952$$

The corresponding damping ratio was:

$$\zeta = 0.0143$$

and the corresponding effective damping coefficient estimate was:

$$k_v = 0.2057 \text{ s}^{-1}$$

This is close to the exponential-envelope estimate:

$$k_v = 0.2116 \text{ s}^{-1}$$

The agreement between these two methods supports the approximate damping estimate. However, the uncertainty in the logarithmic decrement is relatively high. This is mainly because the Hall peak amplitudes are not perfect mechanical amplitude measurements; they are affected by the local path of the magnet over the sensor array.

8.7 Comparison of Damping Models

Three simple envelope models were compared to see which one best described the decay of the Hall peak amplitudes.

The first model was the exponential model, which corresponds to viscous-like damping:

$$A(t) = A_0 e^{-\lambda t}$$

The second model was a linear decay model, which is closer to Coulomb-friction-like behavior:

$$A(t) = A_0 - ct$$

The third model was an inverse decay model, which can represent drag-like behavior:

$$A(t) = \frac{1}{a + bt}$$

The fitted R^2 values were:

Model	R^2
Exponential / viscous-like	0.7111
Linear / Coulomb-friction-like	0.6997
Inverse / drag-like	0.7529

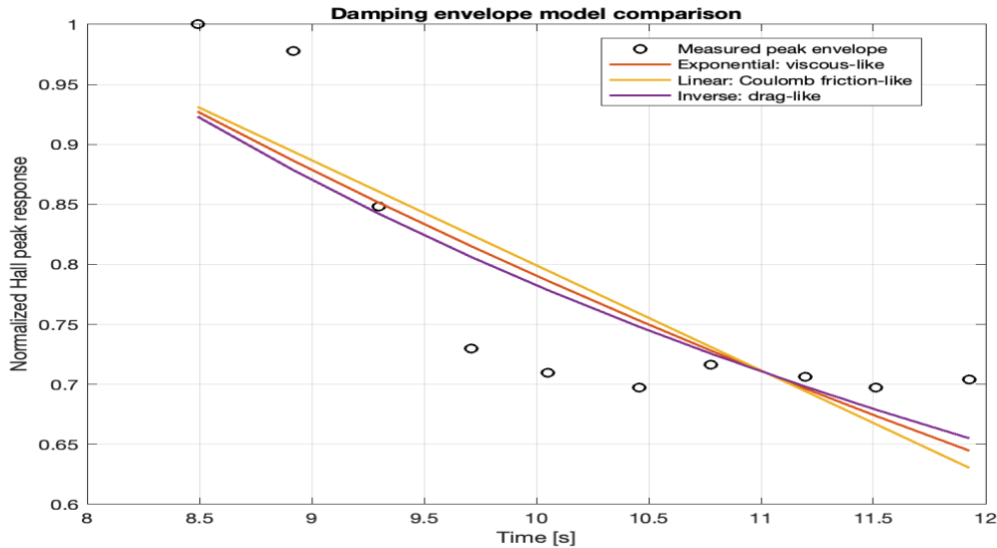


Figure 15. Comparison of exponential, linear, and inverse damping-envelope models fitted to the Hall peak amplitudes.

The inverse/drag-like model gave the highest R^2 , suggesting that air-drag-like effects may have contributed to the decay. However, the difference between the three R^2 values was not large enough to conclude that only one damping mechanism dominated. The more realistic interpretation is that the damping was mixed, with contributions from air resistance, pivot friction, and other mechanical non-idealities.

8.8 Interpretation of the Damping Results

The damping analysis shows that the pendulum was lightly damped. The damping ratio was approximately:

$$\zeta \approx 0.015$$

and the effective damping coefficient per unit inertia was approximately:

$$k_v \approx 0.21 \text{ s}^{-1}$$

These values are consistent with a pendulum that loses energy gradually over time rather than stopping quickly.

However, the result should be interpreted carefully. The Hall sensors measure local magnetic field strength near the sensor array, not the full angular displacement of the pendulum. As a result, the Hall peak envelope is only an indirect indicator of mechanical amplitude. Changes in magnet height, sideways motion, twisting, sensor alignment, and the finite size of the magnet can all affect the measured peak height.

Therefore, the damping result is best described as an **effective damping estimate from the Hall sensor envelope**. It is useful for comparing decay behavior and showing the presence of energy loss, but it should not be treated as an exact mechanical damping constant of the pendulum.

Overall, the Simulink model and the experimental damping analysis support the same physical interpretation: the pendulum motion was nonlinear, lightly damped, and affected by a combination of loss mechanisms. The Hall sensor array successfully detected the crossing events and provided enough information to estimate period, observe decay, and calculate an approximate damping coefficient from real experimental data.

9. Conclusion

This project successfully designed, characterized, and baseline-calibrated a three-Hall-sensor array for detecting the motion of a magnetic pendulum. A neodymium magnet was attached to a pendulum of effective length 0.20 m, and three analog 49E Hall-effect sensors were placed below the pendulum path with 3 cm spacing. The setup converted the pendulum motion into measurable Hall sensor signals that could be used for motion detection, local position estimation, period measurement, and damping analysis.

The Arduino Uno R4 WiFi was configured with 14-bit ADC resolution, giving a voltage resolution of approximately 0.305 mV/count. This significantly reduced the visible quantization effect seen in early tests. A Butterworth filter was tested, but the final analysis relied mainly on higher ADC resolution, baseline subtraction, drift compensation, normalization, and sensor fusion, because these steps preserved the sharp Hall crossing peaks more effectively.

Sensor characterization showed that the three Hall modules had different baseline offsets, confirming the need for baseline calibration before comparing their readings. After correction, the peak magnetic responses were approximately 1.47 to 1.64 V, while the baseline noise was only about 1.19 to 2.00 mV. This gave signal-to-noise ratios greater than 700 for all sensors, and the measured drift was negligible over the short measurement run.

The three-sensor array provided more information than a single Hall sensor. After normalization, the relative responses of H1, H2, and H3 were used to estimate the local magnet position using a weighted centroid method. This reconstruction was valid only within the 6 cm sensor span, corresponding to approximately ± 8.6 degrees around the equilibrium position, so it represented local motion near the sensor array rather than the full pendulum swing.

The measured full period was 0.8766 ± 0.1238 s. This was close to the small-angle theoretical period of 0.8971 s, but lower than the large-angle value of 0.9628 s for the measured 60-degree release. The difference was mainly attributed to Hall peak detection and sensor geometry because the fused signal can contain multiple peaks during one physical crossing.

The damping analysis used the decay of the Hall peak envelope. The exponential fit gave a decay coefficient of 0.1058 s^{-1} , an effective damping ratio of about 0.015, and an effective viscous coefficient per unit inertia of approximately 0.21 s^{-1} . The logarithmic decrement method gave a similar estimate, supporting the result. The comparison of damping models suggested mixed losses rather than one single dominant mechanism.

Overall, the Hall sensor system worked reliably as a characterized and relatively calibrated analog sensor array. It was suitable for detecting pendulum crossings, estimating local position, measuring period, and obtaining an effective damping estimate from real experimental data.

10. Limitations and Future Improvements

10.1 Limitations

The main limitation of the experiment is that the Hall sensors do not directly measure angular displacement. They measure the local magnetic field near the sensor array. Therefore, the Hall peak amplitude depends not only on pendulum amplitude, but also on magnet-sensor distance, magnet orientation, lateral alignment, and the path followed by the magnet during each crossing. For this reason, the damping coefficient obtained from the Hall peak envelope should be treated as an effective damping indicator, not as an exact mechanical damping constant.

The sensor array also had a limited spatial range. The three sensors covered a total span of approximately 6 cm, so the centroid position reconstruction was valid mainly near the central region of the swing. When the pendulum moved outside this range, the array no longer contained enough spatial information to track the true magnet position. This produced apparent clipping near the outer sensor positions and limited the reconstructed angle to approximately ± 8.6 degrees.

The period measurement was affected by the fact that the fused Hall response can contain multiple peaks during one physical crossing. This happens because the magnet passes over H1, H2, and H3 separately. If peak detection identifies these local sensor responses instead of only the central crossing events, the estimated half-period can be underestimated.

Several mechanical factors also affected the measurement. The effective pendulum length had uncertainty because the magnet had finite size and the exact center of mass was not perfectly defined. Although the release angle was measured using a protractor, small errors could still occur due to hand release, pivot alignment, and the initial position of the string. Pivot friction, air resistance, possible twisting, sideways motion, support vibration, and non-planar motion also made the real pendulum different from an ideal mathematical pendulum.

Sensor-related limitations were also present. The three 49E Hall modules had different offsets, noise levels, sensitivities, and drift behavior. These effects were reduced by calibration, baseline subtraction, drift compensation, and normalization, but they cannot be removed perfectly. Small supply-voltage variations and ADC reference uncertainty may also affect voltage values, although the main processing was based on ADC counts and relative changes.

10.2 Future Improvements

The experiment could be improved by repeating the measurements several times for each release angle and averaging the results. This would reduce the effect of random release variation and make the uncertainty in period and damping more reliable.

A larger Hall sensor array could also improve the position reconstruction. Adding more sensors over a wider span would allow the magnet to be tracked over a larger part of the swing instead of only near the center. This would also reduce the clipping problem in the centroid estimate.

The mechanical setup could be improved using a more rigid frame, a low-friction pivot, and a fixed alignment guide for the sensor array. A controlled release mechanism would reduce the uncertainty caused by manual release. Measuring and fixing the magnet-sensor height would also improve repeatability, since the Hall response changes strongly with distance.

The sensor calibration could be made more complete by measuring each Hall sensor response against known magnet positions or known magnetic field values. This would allow the Gaussian response model and centroid method to be calibrated more accurately. An independent measurement system, such as video tracking or an optical encoder, could also be used to validate the Hall-based position and damping estimates.

The period and damping analysis could be improved by separating the individual sensor peaks from the fused signal, or by using only the central sensor for crossing timing while using all three sensors for position estimation. This would reduce the risk of counting multiple peaks during one physical crossing.

Finally, future work could compare different pendulum bob shapes, magnet sizes, release angles, and air-drag conditions. This would make it possible to study how the damping changes when the mechanical losses are intentionally modified.

11. References

- [1] Arduino. “UNO R4 WiFi.” Arduino Documentation. Available: <https://docs.arduino.cc/hardware/uno-r4-wifi>
- [2] Arduino. “ADC Resolution.” Arduino Documentation, UNO R4 WiFi Tutorials. Available: <https://docs.arduino.cc/tutorials/uno-r4-wifi/adc-resolution>
- [3] Honeywell. “SS39ET/SS49E/SS59ET Series Linear Hall-Effect Sensor ICs.” Product Sheet / Datasheet. Available: <https://prod-edam.honeywell.com>
- [4] MathWorks. “MATLAB Documentation.” Available: <https://www.mathworks.com/help/matlab>
- [5] MathWorks. “Serial and USB Communication.” MATLAB Documentation. Available: <https://www.mathworks.com/help/matlab/serial-port-devices.html>
- [6] MathWorks. “Simulink Documentation.” Available: <https://www.mathworks.com/help/simulink>
- [7] D. Halliday, R. Resnick, and J. Walker, *Fundamentals of Physics*. Wiley.
- [8] J. R. Taylor, *Classical Mechanics*. University Science Books.
- [9] D. J. Inman, *Engineering Vibration*. Pearson.

Appendix A. Main Equations Used in the Report

This appendix summarizes the main equations used in the data acquisition, sensor processing, position reconstruction, and damping analysis. They are included here for quick reference.

A.1 ADC-to-voltage conversion

$$0 \leq N \leq 2^{14} - 1 = 16383$$

$$\Delta V = \frac{V_{\text{ref}}}{16383}$$

$$V_i(t) = \frac{N_i(t)}{16383} V_{\text{ref}}$$

A.2 Hall sensor model and corrected response

$$N_i(t) = b_i(t) + g_i B_i(t) + n_i(t)$$

$$S_i(t) = \max(b_i - N_i(t), 0)$$

A.3 Noise, limit of detection, and signal-to-noise ratio

$$\sigma_i = \text{std}(N_i)$$

$$LOD_i = 3\sigma_i$$

$$SNR_i = \frac{S_{i,\text{max}}}{\sigma_i}$$

A.4 Adaptive baseline correction

$$b_i[k] = \alpha b_i[k-1] + (1 - \alpha) N_i[k]$$

$$\alpha = 0.999$$

A.5 Sensor normalization and fused response

$$S_{i,\text{norm}}(t) = \frac{S_i(t)}{S_{i,\text{max}}}$$

$$S_{\text{total}}(t) = S_1(t) + S_2(t) + S_3(t)$$

$$S_{\text{total,norm}}(t) = S_{1,\text{norm}}(t) + S_{2,\text{norm}}(t) + S_{3,\text{norm}}(t)$$

A.6 Local position reconstruction

$$x_{\text{est}}(t) = \frac{x_1 S_{1,\text{norm}}(t) + x_2 S_{2,\text{norm}}(t) + x_3 S_{3,\text{norm}}(t)}{S_{1,\text{norm}}(t) + S_{2,\text{norm}}(t) + S_{3,\text{norm}}(t)}$$

$$\theta_{\text{est}}(t) = \sin^{-1} \left(\frac{x_{\text{est}}(t)}{L} \right)$$

A.7 Pendulum theory and damping

$$\ddot{\theta} + \frac{g}{L} \sin \theta = 0$$

$$T_0 = 2\pi \sqrt{\frac{L}{g}}$$

$$T(\theta_0) = 4 \sqrt{\frac{L}{g}} K \left(\sin^2 \frac{\theta_0}{2} \right)$$

$$A(t) = A_0 e^{-\lambda t}$$

$$\ln A(t) = \ln A_0 - \lambda t$$

$$\zeta \approx \frac{\lambda}{\omega}$$

$$k_v = 2\lambda$$

$$\delta = \ln \left(\frac{A_n}{A_{n+1}} \right)$$

Appendix B. Data Files and Scripts

B.1 Main data files

File	Purpose
hall_sensor_project_baseline_raw.csv	Raw baseline readings used for offset, noise, LOD, and drift estimation.
hall_sensor_project_motion_processed_levels.csv	Processed motion data containing raw ADC values, voltages, corrected responses, normalized responses, estimated baselines, and fused signals.
hall_sensor_project_sensor_characterization.csv	Summary table containing baseline offset, noise, LOD, peak response, SNR, and drift values for H1, H2, and H3.
period_damping_60deg_results.csv	Period and damping results calculated from detected Hall crossing peaks.
centroid_position_estimates.csv	Local magnet position and angle estimates from the weighted centroid method.

B.2 Main scripts and models

B.2.1 Arduino acquisition sketch

```
/*  
  Three Hall Sensor Data Acquisition  
  H1 -> A0  
  H2 -> A1  
  H3 -> A2  
  
  The Arduino reads the three analog Hall sensor outputs,  
  timestamps each sample using micros(), and sends the data  
  to MATLAB as comma-separated values:  
  t_us,H1,H2,H3  
*/  
  
const int H1_PIN = A0;  
const int H2_PIN = A1;  
const int H3_PIN = A2;  
  
// Sampling interval in microseconds  
// 5000 us gives approximately 200 samples per second  
const unsigned long SAMPLE_INTERVAL_US = 5000;  
unsigned long previousSampleTime = 0;  
  
void setup() {  
  Serial.begin(230400);  
}
```

```

// Wait briefly for the serial connection to initialise
delay(1000);

// Configure Arduino Uno R4 WiFi ADC to 14-bit resolution
analogReadResolution(14);

// Optional header line for readability
// MATLAB code can ignore this line if it only stores numeric rows
Serial.println("t_us,H1,H2,H3");
}

void loop() {
    unsigned long currentTime = micros();

    if (currentTime - previousSampleTime >= SAMPLE_INTERVAL_US) {
        previousSampleTime = currentTime;

        int H1 = analogRead(H1_PIN);
        int H2 = analogRead(H2_PIN);
        int H3 = analogRead(H3_PIN);

        Serial.print(currentTime);
        Serial.print(",");
        Serial.print(H1);
        Serial.print(",");
        Serial.print(H2);
        Serial.print(",");
        Serial.println(H3);
    }
}

```

B.2.2 MATLAB acquisition script used to record baseline and motion data, convert ADC counts to voltage, and save CSV files.

```
%% Three Hall Sensor Acquisition and Processing

% Arduino sends CSV lines in the form:
% t_us,H1,H2,H3
%
% This MATLAB script:
% 1. records baseline data,
% 2. records motion data,
% 3. converts ADC counts to voltage,
% 4. estimates baseline offsets,
% 5. performs offset correction and polarity correction,
% 6. applies slow adaptive baseline compensation,
% 7. normalises sensor responses,
% 8. saves CSV files for later analysis.

clc;
clear;
close all;

%% User settings

port = "COM5";           % Change this to your Arduino port
baudRate = 230400;

baselineDuration = 10;    % seconds
motionDuration   = 12;    % seconds

Vref = 5.0;              % nominal ADC reference voltage
ADCmax = 2^14 - 1;        % 14-bit ADC range: 0 to 16383

alpha = 0.999;           % adaptive baseline smoothing factor
baselineUpdateThreshold = 80; % counts; update baseline only when signal is small

%% Connect to Arduino

fprintf("Connecting to Arduino on %s...\n", port);
```

```

arduinoObj = serialport(port, baudRate);
configureTerminator(arduinoObj, "LF");
arduinoObj.Timeout = 2;

pause(1);
flush(arduinoObj);

fprintf("Connection established.\n\n");

%% Record baseline data

fprintf("Recording baseline data for %.1f seconds...\n", baselineDuration);
fprintf("Keep the pendulum still and away from the sensor array.\n\n");

baselineTable = recordHallData(arduinoObj, baselineDuration);

% Convert baseline ADC counts to voltage
baselineTable.H1_V = baselineTable.H1_raw_ADC / ADCmax * Vref;
baselineTable.H2_V = baselineTable.H2_raw_ADC / ADCmax * Vref;
baselineTable.H3_V = baselineTable.H3_raw_ADC / ADCmax * Vref;

% Save baseline data
writetable(baselineTable, "hall_sensor_project_baseline_raw.csv");

fprintf("Baseline data saved as hall_sensor_project_baseline_raw.csv\n\n");

%% Estimate baseline offsets

b1 = median(baselineTable.H1_raw_ADC);
b2 = median(baselineTable.H2_raw_ADC);
b3 = median(baselineTable.H3_raw_ADC);

baselineCounts = [b1, b2, b3];

fprintf("Estimated baseline ADC counts:\n");

```

```

fprintf("H1 = %.2f counts\n", b1);
fprintf("H2 = %.2f counts\n", b2);
fprintf("H3 = %.2f counts\n\n", b3);

%% Record motion data

fprintf("Recording motion data for %.1f seconds...\n", motionDuration);
fprintf("Release the pendulum after starting this recording.\n\n");

motionTable = recordHallData(arduinoObj, motionDuration);

% Convert motion ADC counts to voltage
motionTable.H1_raw_V = motionTable.H1_raw_ADC / ADCmax * Vref;
motionTable.H2_raw_V = motionTable.H2_raw_ADC / ADCmax * Vref;
motionTable.H3_raw_V = motionTable.H3_raw_ADC / ADCmax * Vref;

%% Offset correction and polarity correction

% In this experiment, the magnet polarity caused the Hall readings to drop
% when the magnet approached the sensor. Therefore, the useful magnetic
% response is calculated as baseline minus raw reading.

motionTable.H1_offsetCorrected_counts = max(b1 - motionTable.H1_raw_ADC, 0);
motionTable.H2_offsetCorrected_counts = max(b2 - motionTable.H2_raw_ADC, 0);
motionTable.H3_offsetCorrected_counts = max(b3 - motionTable.H3_raw_ADC, 0);

%% Adaptive baseline drift compensation

N = height(motionTable);

estimatedBaseline = zeros(N, 3);
driftCorrected = zeros(N, 3);

b = baselineCounts;

```

```

rawADC = [
    motionTable.H1_raw_ADC, ...
    motionTable.H2_raw_ADC, ...
    motionTable.H3_raw_ADC
];

for k = 1:N

    % Temporary magnetic response using the current baseline estimate
    tempResponse = max(b - rawADC(k, :), 0);
    totalResponse = sum(tempResponse);

    % Update baseline only when the magnet is probably far from the sensors
    if totalResponse < baselineUpdateThreshold
        b = alpha * b + (1 - alpha) * rawADC(k, :);
    end

    estimatedBaseline(k, :) = b;
    driftCorrected(k, :) = max(b - rawADC(k, :), 0);

end

motionTable.H1_estimatedBaseline_counts = estimatedBaseline(:, 1);
motionTable.H2_estimatedBaseline_counts = estimatedBaseline(:, 2);
motionTable.H3_estimatedBaseline_counts = estimatedBaseline(:, 3);

motionTable.H1_driftCorrected_counts = driftCorrected(:, 1);
motionTable.H2_driftCorrected_counts = driftCorrected(:, 2);
motionTable.H3_driftCorrected_counts = driftCorrected(:, 3);

%% Sensitivity normalisation

H1max = max(motionTable.H1_driftCorrected_counts);
H2max = max(motionTable.H2_driftCorrected_counts);
H3max = max(motionTable.H3_driftCorrected_counts);

```

```

motionTable.H1_normalized = motionTable.H1_driftCorrected_counts / H1max;
motionTable.H2_normalized = motionTable.H2_driftCorrected_counts / H2max;
motionTable.H3_normalized = motionTable.H3_driftCorrected_counts / H3max;

%% Fused magnetic response

motionTable.TotalResponse_counts = ...
    motionTable.H1_driftCorrected_counts + ...
    motionTable.H2_driftCorrected_counts + ...
    motionTable.H3_driftCorrected_counts;

motionTable.TotalResponse_normalized = ...
    motionTable.TotalResponse_counts / max(motionTable.TotalResponse_counts);

%% Save processed motion data
writetable(motionTable, "hall_sensor_project_motion_processed_levels.csv");
fprintf("Motion data saved as hall_sensor_project_motion_processed_levels.csv\n\n");

%% Sensor characterisation table

noiseCounts = [
    std(baselineTable.H1_raw_ADC), ...
    std(baselineTable.H2_raw_ADC), ...
    std(baselineTable.H3_raw_ADC)
];

noiseVoltage_mV = noiseCounts / ADCmax * Vref * 1000;

LOD_counts = 3 * noiseCounts;
LOD_mV = 3 * noiseVoltage_mV;

peakResponse_counts = [H1max, H2max, H3max];
peakResponse_mV = peakResponse_counts / ADCmax * Vref * 1000;

```

```

SNR = peakResponse_counts ./ noiseCounts;

sensorNames = ["H1"; "H2"; "H3"];

characterisationTable = table( ...
    sensorNames, ...
    baselineCounts(:), ...
    baselineCounts(:) / ADCmax * Vref, ...
    noiseCounts(:), ...
    noiseVoltage_mV(:), ...
    LOD_counts(:), ...
    LOD_mV(:), ...
    peakResponse_counts(:), ...
    peakResponse_mV(:), ...
    SNR(:), ...
    'VariableNames', { ...
        'Sensor', ...
        'Baseline_ADC_count', ...
        'Baseline_voltage_V', ...
        'Noise_std_counts', ...
        'Noise_std_mV', ...
        'LOD_counts', ...
        'LOD_mV', ...
        'Peak_response_counts', ...
        'Peak_response_mV', ...
        'SNR' ...
    } ...
);

writetable(characterisationTable, "hall_sensor_project_sensor_characterization.csv");

fprintf("Sensor characterisation saved as hall_sensor_project_sensor_characterization.csv\n");
%% Close serial connection
clear arduinoObj;
fprintf("Acquisition complete.\n");

```

```

%% Local function for recording serial Hall sensor data

function dataTable = recordHallData(serialObj, durationSeconds)

    tLocal = [];
    tArduino_us = [];
    H1 = [];
    H2 = [];
    H3 = [];

    flush(serialObj);
    pause(0.2);
    startTime = tic;
    while toc(startTime) < durationSeconds

        try
            line = strtrim(readline(serialObj));
        catch
            continue;
        end

        values = str2double(split(line, ","));

        % Ignore header lines, incomplete lines, and non-numeric lines
        if numel(values) ~= 4 || any(isnan(values))
            continue;
        end

        tLocal(end + 1, 1) = toc(startTime);
        tArduino_us(end + 1, 1) = values(1);
        H1(end + 1, 1) = values(2);
        H2(end + 1, 1) = values(3);
        H3(end + 1, 1) = values(4);
    end

    % Convert Arduino timestamp to seconds starting from zero
    tArduino_s = (tArduino_us - tArduino_us(1)) * 1e-6;

    dataTable = table( ...

```

```

        tLocal, ...
        tArduino_s, ...
        H1, ...
        H2, ...
        H3, ...
        'VariableNames', { ...
            't_s', ...
            't_arduino_s', ...
            'H1_raw_ADC', ...
            'H2_raw_ADC', ...
            'H3_raw_ADC' ...
        } ...
    );

end

```

B.2.3 MATLAB weighted-centroid reconstruction script used to estimate the local magnet position from the three normalised Hall sensor responses.

```

clear; clc; close all;

%% =====
% WEIGHTED CENTROID POSITION RECONSTRUCTION
% Final version:
% - One position plot
% - One GIF animation
% =====

%% Load processed CSV
fileName = "hall_sensor_project_motion_processed_levels.csv";
data = readtable(fileName);

%% Geometry
L = 0.20; % pendulum length [m]
xSensors = [-0.03, 0, 0.03]; % H1, H2, H3 positions [m]

```

```

%% Time
t = data.t_s;

%% Use normalized Hall responses
% These should already come from your main processing script
S1 = data.H1_normalized;
S2 = data.H2_normalized;
S3 = data.H3_normalized;

S = [S1, S2, S3];

%% Total response
Stotal = sum(S, 2);

%% Weighted centroid estimate
xEst = (S(:,1)*xSensors(1) + S(:,2)*xSensors(2) + S(:,3)*xSensors(3)) ./ (Stotal + eps);

%% Threshold for valid position estimate
% The centroid is only meaningful when the magnet is close enough
% to the sensor array.
threshold = 0.08;
valid = Stotal > threshold;

xEstMasked = xEst;
xEstMasked(~valid) = NaN;

%% Light smoothing for cleaner visualization
xEstSmooth = smoothdata(xEstMasked, "movmedian", 5);

%% =====
% PLOT: WEIGHTED CENTROID POSITION
% =====

figure("Color", "w", "Position", [100 100 950 550]);

```

```

hold on;

plot(t, xEstSmooth*100, "k", "LineWidth", 1.8);

yline(xSensors(1)*100, "--", "H1 position", ...
      "Color", [0.35 0.35 0.35], "LineWidth", 1.0, ...
      "LabelHorizontalAlignment", "left");

yline(xSensors(2)*100, "--", "H2 position", ...
      "Color", [0.35 0.35 0.35], "LineWidth", 1.0, ...
      "LabelHorizontalAlignment", "left");

yline(xSensors(3)*100, "--", "H3 position", ...
      "Color", [0.35 0.35 0.35], "LineWidth", 1.0, ...
      "LabelHorizontalAlignment", "left");

xlabel("Time [s]", "FontSize", 13);
ylabel("Estimated magnet position [cm]", "FontSize", 13);
title("Weighted centroid estimate of magnet position", ...
      "FontSize", 15, "FontWeight", "bold");

grid on;
box on;
ylim([-4 4]);
xlim([min(t) max(t)]);

set(gca, "FontSize", 12, ...
      "LineWidth", 1.0, ...
      "Color", [0.98 0.98 0.98]);

saveas(gcf, "centroid_estimated_position.png");
saveas(gcf, "centroid_estimated_position.pdf");

%% =====
% GIF: POSITION TRACING OVER SENSOR ARRAY

```

```

% =====

gifFile = "weighted_centroid_magnet_animation.gif";

figure("Color", "w", "Position", [100 100 850 700]);

frameStep = 1; % increase this if you want a smaller GIF

for k = 1:frameStep:length(t)

    clf;

    %% Top panel: sensor array and estimated magnet position
    subplot(2,1,1);
    hold on;

    % Sensor rail
    plot([-4 4], [0 0], "k-", "LineWidth", 1.4);

    % Sensors
    scatter(xSensors*100, [0 0 0], 260, ...
        [0.00 0.45 0.74;
         0.85 0.33 0.10;
         0.47 0.67 0.19], ...
        "filled");

    text(xSensors(1)*100, -0.32, "H1", "HorizontalAlignment", "center", "FontSize", 13);
    text(xSensors(2)*100, -0.32, "H2", "HorizontalAlignment", "center", "FontSize", 13);
    text(xSensors(3)*100, -0.32, "H3", "HorizontalAlignment", "center", "FontSize", 13);

    if valid(k)
        scatter(xEstSmooth(k)*100, 0.70, 420, [1 0 1], "filled");
        text(xEstSmooth(k)*100, 1.02, "Estimated magnet", ...
            "HorizontalAlignment", "center", ...
            "Color", [1 0 1], ...

```

```

        "FontSize", 15);

    else

        text(0, 0.90, "Magnet outside reliable sensing region", ...
            "HorizontalAlignment", "center", ...
            "Color", [0.35 0.35 0.35], ...
            "FontSize", 15);

    end

    xlim([-4 4]);
    ylim([-0.8 1.5]);
    grid on;
    box on;

    xlabel("Position along sensor array [cm]", "FontSize", 13);
    title(sprintf("Weighted centroid reconstruction, t = %.2f s", t(k)), ...
        "FontSize", 15, "FontWeight", "bold");

    text(0, -0.62, ...
        "Vertical placement of the magenta marker is only for display; only the x-position is physical.", ...
        "HorizontalAlignment", "center", ...
        "FontSize", 10, ...
        "Color", [0.30 0.30 0.30]);

    set(gca, "FontSize", 12, ...
        "LineWidth", 1.0, ...
        "Color", [0.98 0.98 0.98]);

    %% Bottom panel: instantaneous normalized responses
    subplot(2,1,2);
    bar(1:3, S(k,:), 0.55, "FaceColor", [0.10 0.45 0.75]);

    ylim([0 1.1]);
    xlim([0.3 3.7]);
    xticks(1:3);

```

```

xticklabels({"H1", "H2", "H3"});

ylabel("Normalized response", "FontSize", 13);
title("Instantaneous normalized Hall responses", ...
      "FontSize", 15, "FontWeight", "bold");

grid on;
box on;

set(gca, "FontSize", 12, ...
      "LineWidth", 1.0, ...
      "Color", [0.98 0.98 0.98]);

drawnow;

%% Write GIF frame
frame = getframe(gcf);
im = frame2im(frame);
[A, map] = rgb2ind(im, 256);

if k == 1
    imwrite(A, map, gifFile, "gif", "LoopCount", Inf, "DelayTime", 0.05);
else
    imwrite(A, map, gifFile, "gif", "WriteMode", "append", "DelayTime", 0.05);
end
end

disp("Saved files:");
disp("centroid_estimated_position.png");
disp("centroid_estimated_position.pdf");
disp(gifFile);

```

B.2.4 MATLAB period and damping analysis script.

```
clear; clc; close all;

%% Period and damping analysis from fused Hall response

fileName = "hall_sensor_project_motion_processed_levels.csv";
data = readtable(fileName);

%% Settings

L = 0.20;           % pendulum length [m]
g = 9.81;           % gravitational acceleration [m/s^2]
releaseAngle_deg = 60;

smoothWindow = 21;
minPeakProminence = 400;
minPeakDistance_s = 0.25;

dampingStartTime = 8.4; % use the later part where the decay envelope is clearer

%% Time and fused Hall response

if ismember("t_arduino_s", data.Properties.VariableNames)
    t = data.t_arduino_s;
else
    t = data.t_s;
end

S_fused = data.TotalResponse_counts;

%% Smooth the fused signal for peak detection

S_smooth = smoothdata(S_fused, "sgolay", smoothWindow);
```

```

%% Detect Hall crossing peaks

[peakAmp, peakTime] = findpeaks(S_smooth, t, ...
    "MinPeakProminence", minPeakProminence, ...
    "MinPeakDistance", minPeakDistance_s);

numPeaks = length(peakTime);

if numPeaks < 4
    error("Too few peaks were detected. Try reducing MinPeakProminence or MinPeakDistance.");
end

%% Period estimate

halfPeriods = diff(peakTime);

meanHalfPeriod = mean(halfPeriods);
stdHalfPeriod = std(halfPeriods);

T_measured = 2 * meanHalfPeriod;
T_uncertainty = 2 * stdHalfPeriod;

f_measured = 1 / T_measured;

%% Theoretical periods

theta0 = deg2rad(releaseAngle_deg);

T_small = 2*pi*sqrt(L/g);

m = sin(theta0/2)^2;
K = ellipke(m);
T_large = 4*sqrt(L/g)*K;

error_small_percent = (T_measured - T_small) / T_small * 100;

```

```

error_large_percent = (T_measured - T_large) / T_large * 100;

%% Select peaks for damping-envelope fitting

dampingMask = peakTime >= dampingStartTime;

dampTime = peakTime(dampingMask);
dampAmp = peakAmp(dampingMask);

if length(dampTime) < 4
    error("Too few damping peaks were selected. Try lowering dampingStartTime.");
end

% Normalise the selected peak amplitudes for envelope comparison.
A = dampAmp / max(dampAmp);

% Use local time for fitting.
tau = dampTime - dampTime(1);

%% Model 1: exponential decay, viscous-like damping

%  $A(t) = A_0 \exp(-\lambda t)$ 
logA = log(A);

p_exp = polyfit(tau, logA, 1);

lambda = -p_exp(1);
A0_exp = exp(p_exp(2));

A_exp = A0_exp * exp(-lambda * tau);

R2_exp = rsquared(A, A_exp);

%% Model 2: linear decay, Coulomb-friction-like damping

```

```

% A(t) = A0 - c t
p_lin = polyfit(tau, A, 1);

c_lin = -p_lin(1);
A0_lin = p_lin(2);

A_lin = polyval(p_lin, tau);

R2_lin = rsquared(A, A_lin);

%% Model 3: inverse decay, drag-like damping

% A(t) = 1 / (a + b t)
p_inv = polyfit(tau, 1 ./ A, 1);

b_inv = p_inv(1);
a_inv = p_inv(2);

A_inv = 1 ./ (a_inv + b_inv * tau);

R2_inv = rsquared(A, A_inv);

%% Effective damping estimates

omega0 = sqrt(g/L);
omega_measured = 2*pi / T_measured;
omega_large = 2*pi / T_large;

zeta_using_small_angle = lambda / omega0;
zeta_using_measured_period = lambda / omega_measured;
zeta_using_large_angle = lambda / omega_large;

k_v_exp = 2 * lambda;

%% Logarithmic decrement

```

```
% Hall crossings occur twice per full oscillation, so every second peak is
% compared to estimate decrement over one full period.
```

```
if length(A) >= 3
```

```
    deltaValues = log(A(1:end-2) ./ A(3:end));
```

```
else
```

```
    deltaValues = log(A(1:end-1) ./ A(2:end));
```

```
end
```

```
deltaMean = mean(deltaValues, "omitnan");
```

```
deltaStd = std(deltaValues, "omitnan");
```

```
zeta_log = deltaMean / sqrt((2*pi)^2 + deltaMean^2);
```

```
lambda_log = deltaMean / T_measured;
```

```
k_v_log = 2 * lambda_log;
```

```
%% Summary tables
```

```
periodTable = table( ...
```

```
    numPeaks, ...
```

```
    meanHalfPeriod, ...
```

```
    stdHalfPeriod, ...
```

```
    T_measured, ...
```

```
    T_uncertainty, ...
```

```
    f_measured, ...
```

```
    T_small, ...
```

```
    T_large, ...
```

```
    error_small_percent, ...
```

```
    error_large_percent, ...
```

```
    'VariableNames', { ...
```

```
        'DetectedPeaks', ...
```

```
        'MeanHalfPeriod_s', ...
```

```
        'StdHalfPeriod_s', ...
```

```
        'MeasuredPeriod_s', ...
```

```

        'PeriodUncertainty_s', ...
        'MeasuredFrequency_Hz', ...
        'SmallAnglePeriod_s', ...
        'LargeAnglePeriod_s', ...
        'ErrorVsSmallAngle_percent', ...
        'ErrorVsLargeAngle_percent' ...
    });

modelTable = table( ...
    ["Exponential / viscous-like"; ...
    "Linear / Coulomb-friction-like"; ...
    "Inverse / drag-like"], ...
    [R2_exp; R2_lin; R2_inv], ...
    ["lambda = " + string(lambda) + " s^-1"; ...
    "c = " + string(c_lin); ...
    "a = " + string(a_inv) + ", b = " + string(b_inv)], ...
    'VariableNames', {'Model', 'R2', 'FitParameters'});

dampingTable = table( ...
    lambda, ...
    k_v_exp, ...
    zeta_using_small_angle, ...
    zeta_using_measured_period, ...
    zeta_using_large_angle, ...
    deltaMean, ...
    deltaStd, ...
    zeta_log, ...
    k_v_log, ...
    'VariableNames', { ...
        'ExponentialLambda_s_inv', ...
        'EffectiveKv_exp_s_inv', ...
        'Zeta_usingSmallAngle', ...
        'Zeta_usingMeasuredPeriod', ...
        'Zeta_usingLargeAngle', ...
        'MeanLogDecrement', ...
    }

```

```

        'StdLogDecrement', ...
        'Zeta_logDecrement', ...
        'EffectiveKv_log_s_inv' ...
    });

disp(" ");
disp("PERIOD RESULTS");
disp(periodTable);

disp(" ");
disp("DAMPING MODEL COMPARISON");
disp(modelTable);

disp(" ");
disp("DAMPING ESTIMATES");
disp(dampingTable);

writetable(periodTable, "damping_period_results.csv");
writetable(modelTable, "damping_model_comparison_results.csv");
writetable(dampingTable, "damping_estimates_results.csv");

%% Plot 1: detected Hall crossing peaks

figure("Color", "w", "Position", [100 100 1050 600]);
hold on;

plot(t, S_fused, "Color", [0.75 0.75 0.75], "LineWidth", 0.9);
plot(t, S_smooth, "k", "LineWidth", 1.4);

plot(peakTime, peakAmp, "ro", ...
     "MarkerSize", 6, ...
     "MarkerFaceColor", "w", ...
     "LineWidth", 1.3);

xlabel("Time [s]", "FontSize", 13);

```

```

ylabel("Fused magnetic response [ADC counts]", "FontSize", 13);
title("Detected Hall crossing peaks from fused magnetic response", ...
      "FontSize", 15, "FontWeight", "bold");

legend("Raw fused response", "Smoothed response", "Detected peaks", ...
      "Location", "best");

grid on;
box on;

set(gca, "FontSize", 12, ...
      "LineWidth", 1.0, ...
      "Color", [0.98 0.98 0.98]);

exportgraphics(gcf, "damping_plot1_peak_detection.png", "Resolution", 300);
exportgraphics(gcf, "damping_plot1_peak_detection.pdf", "ContentType", "vector");

%% Plot 2: damping model comparison

fitTime = linspace(min(dampTime), max(dampTime), 400);
fitTau = fitTime - dampTime(1);

fitExp = A0_exp * exp(-lambda * fitTau);
fitLin = A0_lin - c_lin * fitTau;
fitInv = 1 ./ (a_inv + b_inv * fitTau);

figure("Color", "w", "Position", [100 100 1050 600]);
hold on;

plot(dampTime, A, "ko", ...
      "MarkerSize", 7, ...
      "MarkerFaceColor", "w", ...
      "LineWidth", 1.3);

plot(fitTime, fitExp, ...

```

```

"LineWidth", 2.1);

plot(fitTime, fitLin, ...
     "LineWidth", 2.1);

plot(fitTime, fitInv, ...
     "LineWidth", 2.1);

xlabel("Time [s]", "FontSize", 13);
ylabel("Normalised Hall peak response", "FontSize", 13);
title("Damping envelope model comparison", ...
      "FontSize", 15, "FontWeight", "bold");

legend( ...
      "Measured Hall peak envelope", ...
      sprintf("Exponential / viscous-like, R^2 = %.4f", R2_exp), ...
      sprintf("Linear / Coulomb-friction-like, R^2 = %.4f", R2_lin), ...
      sprintf("Inverse / drag-like, R^2 = %.4f", R2_inv), ...
      "Location", "best");

grid on;
box on;

set(gca, "FontSize", 12, ...
      "LineWidth", 1.0, ...
      "Color", [0.98 0.98 0.98]);

exportgraphics(gcf, "damping_plot2_model_comparison.png", "Resolution", 300);
exportgraphics(gcf, "damping_plot2_model_comparison.pdf", "ContentType", "vector");

%% Print report-ready values

disp(" ");
disp("SUMMARY FOR REPORT");
disp("=====");

```

```

fprintf("Number of detected Hall crossing peaks = %d\n", numPeaks);
fprintf("Mean half-period = %.4f s\n", meanHalfPeriod);
fprintf("Standard deviation of half-period = %.4f s\n", stdHalfPeriod);
fprintf("Measured full period = %.4f ± %.4f s\n", T_measured, T_uncertainty);
fprintf("Measured frequency = %.4f Hz\n", f_measured);

fprintf("\nSmall-angle theoretical period = %.4f s\n", T_small);
fprintf("Large-angle theoretical period at %.0f degrees = %.4f s\n", releaseAngle_deg, T_large);

fprintf("\nError relative to small-angle theory = %.2f %%\n", error_small_percent);
fprintf("Error relative to large-angle theory = %.2f %%\n", error_large_percent);

fprintf("\nExponential decay coefficient lambda = %.4f s^-1\n", lambda);
fprintf("Effective viscous coefficient per unit inertia, k_v = %.4f s^-1\n", k_v_exp);

fprintf("\nDamping ratios:\n");
fprintf("Using small-angle omega0 = %.4f\n", zeta_using_small_angle);
fprintf("Using measured angular frequency = %.4f\n", zeta_using_measured_period);
fprintf("Using large-angle angular frequency = %.4f\n", zeta_using_large_angle);

fprintf("\nLogarithmic decrement = %.4f ± %.4f\n", deltaMean, deltaStd);
fprintf("Log-decrement damping ratio = %.4f\n", zeta_log);
fprintf("Log-decrement k_v = %.4f s^-1\n", k_v_log);

fprintf("\nModel comparison R^2 values:\n");
fprintf("Exponential / viscous-like = %.4f\n", R2_exp);
fprintf("Linear / Coulomb-friction-like = %.4f\n", R2_lin);
fprintf("Inverse / drag-like = %.4f\n", R2_inv);

disp(" ");
disp("Saved files:");
disp("damping_plot1_peak_detection.png");
disp("damping_plot1_peak_detection.pdf");
disp("damping_plot2_model_comparison.png");

```

```
disp("damping_plot2_model_comparison.pdf");  
disp("damping_period_results.csv");  
disp("damping_model_comparison_results.csv");  
disp("damping_estimates_results.csv");
```

```
%% Local function
```

```
function R2 = rsquared(y, yFit)
```

```
    y = y(:);
```

```
    yFit = yFit(:);
```

```
    SS_res = sum((y - yFit).^2);
```

```
    SS_tot = sum((y - mean(y)).^2);
```

```
    R2 = 1 - SS_res / SS_tot;
```

```
end
```